



Faculty of Engineering and Technology

Computer Science Department

Data Science and Analytics - COMP4381

**DelayGuide: A Predictive Analytics System for Flight Delay  
Analysis and Prediction**

**Prepared by:**

Raheeq Mousa - 1220515

Aya Abd-Alkarim - 1220020

Zaid Mousa - 1221833

**Instructor:** Ahmad Sabbah

## **Abstract**

Flight delays are a common problem in global air travel and they cause higher costs for airlines and inconvenience for passengers. This project builds a full data science pipeline to study and predict flight delays using data from airports, and weather conditions across at least five countries. The data is collected from reliable aviation sources, cleaned, and processed to create a dataset with more than 1000 records and at least 15 important features such as flight schedules, airport information, and weather factors.

A machine learning approach is used to predict whether a flight will be delayed or not (binary classification). Different models will be tested and compared to find the best performing one. The main goal is not only to build a prediction model but also to understand the key factors that cause delays. The results help show how data science can be used to improve planning and decision-making in the airline industry.

# Contents

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1. Introduction .....                                             | 4  |
| 2. Background .....                                               | 6  |
| 2.1 The Nature and Propagation of Flight Delays. ....             | 6  |
| 2.2 Causes of Flight Delays. ....                                 | 6  |
| 2.3 Machine Learning in Flight Delay Prediction. ....             | 7  |
| 2.4 Challenges in Building Predictive Models.....                 | 7  |
| 3. Domain explanation .....                                       | 8  |
| 4. Algorithm Background .....                                     | 9  |
| 4.1 Logistic Regression .....                                     | 9  |
| 4.2 Decision Tree .....                                           | 10 |
| 4.3 Random Forest .....                                           | 10 |
| 4.4 XGBoost.....                                                  | 11 |
| 4.5 Model Evaluation Metrics.....                                 | 11 |
| 4.6 Handling Class Imbalance Using Class Weights .....            | 12 |
| 5. Literature Review .....                                        | 12 |
| 5.1 Overview of Selected Studies .....                            | 12 |
| 5.2 Comparative Summary Table .....                               | 17 |
| 6. Dataset description .....                                      | 18 |
| 7. Data source explanation .....                                  | 19 |
| 8. Data cleaning and preprocessing .....                          | 19 |
| 8.1 Sampling data .....                                           | 20 |
| 8.2 Merging Airport Coordinate Data .....                         | 21 |
| 8.3 Validate datatypes.....                                       | 23 |
| 8.4 Solving Inconsistency .....                                   | 27 |
| 8.5 Duplicates.....                                               | 29 |
| 8.6 Missing Data.....                                             | 29 |
| 8.7 Outliers .....                                                | 30 |
| 8.8 Introduce features .....                                      | 31 |
| 9. Data summarization .....                                       | 36 |
| 9.1 Review Dataset after cleaning to do summarization on it ..... | 36 |
| 9.1.1 Print the number of origin and destination cities.....      | 38 |
| 9.1.2 Drop unnamed column .....                                   | 39 |
| 9.1.3 Mode.....                                                   | 39 |
| 9.2 Categorical Data Analysis.....                                | 40 |

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| 9.2.1.1 Check which season is the most season that may have delayed .....            | 40 |
| 9.2.1.2 find the percentage of delay for each season .....                           | 40 |
| 9.2.2 Which period of the day have the most delayed .....                            | 41 |
| 9.2.3 which city have the most delayed flights .....                                 | 42 |
| 9.3.4 which month have the most delayed flights .....                                | 42 |
| 9.3 Numerical Data Analysis .....                                                    | 43 |
| 9.3.1 Feature Correlation Analysis .....                                             | 43 |
| for this interpretation I try to find the most strong and weak relationships .....   | 43 |
| 9.3.2 Interpretation of describe Results .....                                       | 44 |
| 9.3.3 Feature Correlation Analysis how every numeric column relates to delayed ..... | 45 |
| 9.3.4 Calculate the delay percentage for each airline. ....                          | 45 |
| 9.3.5 Calculate the origin delay percentage. ....                                    | 46 |
| 9.3.6 Calculate the number of flight observations in each origin .....               | 47 |
| 10. Visualizations and interpretation .....                                          | 48 |
| 10.1 Data Distribution .....                                                         | 48 |
| 10.2 Average Delayed Percentage by Month .....                                       | 50 |
| 10.3 Average Delayed Percentage by Day of Week .....                                 | 51 |
| 10.5 Distance vs Delay .....                                                         | 53 |
| 10.6 Top 10 Airports with Highest Delay Percentage .....                             | 54 |
| 10.7 Top 10 Airlines with Highest Delay .....                                        | 54 |
| 10.8 Correlation Heatmap .....                                                       | 55 |
| 10.9 Delay Heatmap (Month vs Day) .....                                              | 58 |
| 11. References .....                                                                 | 60 |

## Acronyms and Abbreviations

|         |                                                             |
|---------|-------------------------------------------------------------|
| ML      | Machine Learning                                            |
| KNN     | K-Nearest Neighbor                                          |
| SVM     | Support Vector Machine                                      |
| RF      | Random Forest                                               |
| DT      | Decision Tree                                               |
| LR      | Logistic Regression                                         |
| GBT     | Gradient Boosted Tree                                       |
| NB      | Naïve Bayes                                                 |
| GNB     | Gaussian Naïve Bayes                                        |
| XGBoost | Extreme Gradient Boosting                                   |
| SMOTE   | Synthetic Minority Over-sampling Technique                  |
| SHAP    | SHapley Additive exPlanations                               |
| ANN     | Artificial Neural Network                                   |
| BTS     | Bureau of Transportation Statistics                         |
| IATA    | International Air Transport Association                     |
| ADS-B   | Automatic Dependent Surveillance–Broadcast                  |
| ROC-AUC | Receiver Operating Characteristic – Area Under Curve        |
| DBSCAN  | Density-Based Spatial Clustering of Applications with Noise |
| ADASYN  | Adaptive Synthetic Sampling                                 |
| ROS     | Random Oversampling                                         |
| XAI     | Explainable Artificial Intelligence                         |
| COA     | Coyote Optimization Algorithm                               |

## Table of Figure

|                                                                      |   |
|----------------------------------------------------------------------|---|
| Figure 1: Logistic Regression Explanation Figure (Source: [7]) ..... | 5 |
| Figure 2: Decision Tree explanation Figure (Source: [8]) .....       | 6 |
| Figure 3: Random Forest Explanation Figure (Source: [9]) .....       | 7 |

# 1. Introduction

As global air transportation continues to grow, flight delays have become a major concern for both the aviation industry and researchers. Flight delays remain frequent and have significant impacts on passengers, airlines, and airport operations. Passengers often face missed connections, increased travel time, and additional costs, while airlines experience higher operational expenses and reduced efficiency. At the airport level, delays negatively affect runway utilization, terminal performance, and overall system capacity.

Beyond operational issues, recent studies show that flight delays also have important economic consequences. Delays can influence airfare pricing and flight frequency, as airlines adjust their schedules and pricing strategies in response to persistent disruptions. This indicates that flight delays are not only a logistical challenge but also an economic factor that affects the structure and efficiency of the aviation market.

The causes of flight delays are complex and interconnected. Key factors include adverse weather conditions, airport congestion, air traffic control restrictions, aircraft technical problems, and airline operational decisions. In addition, recent research highlights that delays are not isolated events; instead, they can propagate through the flight network. A delay in one airport may affect connected flights and airports, leading to a cascading effect that increases overall system disruption.

Because of this complexity, accurately predicting flight delays is a challenging task. Traditional statistical methods are often limited in capturing nonlinear relationships and dependencies between multiple variables. As a result, machine learning techniques have become widely used in recent studies due to their ability to analyze large-scale aviation data and identify hidden patterns that contribute to delays.

In this project, we develop a data science framework to analyze and predict flight delays across multiple countries. The framework includes data collection, preprocessing, exploratory data analysis, and machine learning-based modeling. The main objective is to build a predictive system that not only estimates flight delays but also helps identify the most important factors influencing them. This contributes to a better understanding of delay behavior and supports more informed decision-making in airline and airport operations [\[1\]](#).

## **2. Background**

### **2.1 The Nature and Propagation of Flight Delays.**

Flight delays are not something new in the aviation industry. For many years, airlines and airports have fought with keeping flights on schedule. Even a small delay in one flight can affect many others during the day. For example, if an aircraft arrives late, it may depart late again for its next trip. This creates a chain reaction where delays continue to spread across the schedule [\[2\]](#). Because of this cascading effect, managing flight operations is more complex than it may appear. Forecasting flight delays can enhance airline efficiency and improve passenger satisfaction, ultimately contributing to positive economic outcomes.

### **2.2 Causes of Flight Delays.**

There are many reasons why flights get delayed. Weather is one of the main factors, especially during winter or storm seasons. Heavy rain, snow, fog, or strong winds can slow down takeoffs and landings. Busy airports also experience congestion, where too many flights are scheduled at the same time, leading to runway or gate shortages. Other reasons include technical maintenance issues,

crew availability, air traffic control restrictions, and operational decisions made by airlines. Since these factors vary across countries, delay patterns are different depending on location and airport size.

Research confirms that weather and operational factors are the two most common causes of delays across different countries and airports [\[3\]](#). Studies also found that airport congestion and air traffic control restrictions play a significant role, especially at large and busy hub airports [\[2\]](#).

### **2.3 Machine Learning in Flight Delay Prediction.**

In recent years, researchers have started using machine learning to better understand and predict flight delays. With the availability of large datasets, it is now possible to analyze millions of flight records and identify patterns that were difficult to detect before. Instead of relying only on simple statistical methods, machine learning models can learn from complex relationships between features such as airport characteristics, time of travel, and weather conditions [\[3\]](#). Studies have shown that ensemble methods such as Random Forest and XGBoost consistently achieve strong predictive performance on flight delay classification tasks [\[2\]\[3\]](#).

### **2.4 Challenges in Building Predictive Models.**

However, predicting delays is still challenging. Flight data often contains missing values, imbalanced categories, and inconsistencies between sources. Some types of delays happen much more frequently than others, which can affect model performance if not handled properly [\[4\]](#). A particularly significant challenge is class imbalance, since in most real-world datasets, the number of on-time flights is much higher than delayed flights. This can cause models to be biased toward predicting the majority class and reduce performance on delayed flights [\[2\]\[3\]](#).

To address this issue, techniques such as SMOTE (Synthetic Minority Over-sampling Technique) oversampling or class-weight adjustment are often used during model training. For these reasons, careful data cleaning, preprocessing, and feature engineering are essential before building any prediction model [\[2\]](#).

### **3. Domain explanation**

This project falls within the aviation industry which includes the activities surrounding mechanical flight and the aircraft industry. The main functionality for it is interconnected sectors airlines, manufacturers, airports, and service providers so this is why the aviation domain produces large amounts of data, like flight schedules from the airlines, airport information, and weather conditions from the airports. This data is important for analyzing system performance and identifying issues such as flight delays.

Flight delays are a major challenge in this domain due to their negatively impact costs, customer satisfaction, and revenue in the aviation industry. Since delays are influenced by multiple interconnected factors, analyzing them requires a ML with a data-driven approach.

Data science techniques can help a lot to analyze flight data, detect patterns, know the main reasons of happening, and support decision-making. In this project, these techniques are applied to better understand and predict flight delays.

## 4. Algorithm Background

This project applies supervised machine learning for binary classification, where the goal is to predict whether a given flight will be delayed (1) or on time (0). Several algorithms are considered based on their proven performance in related studies. This section provides the theoretical foundation for each algorithm and explains the techniques used to handle class imbalance.

### 4.1 Logistic Regression

Logistic Regression is a classification algorithm used for binary outcomes. It estimates the probability that a given input belongs to a particular class using a logistic (sigmoid) function. Given a set of input features  $x$ , the model computes a weighted linear combination of the features and passes it through the sigmoid function to produce a probability between 0 and 1. A threshold (typically 0.5) is then applied to classify the output as delayed or on time. Also, it is commonly used as a baseline model in flight delay prediction studies due to its simplicity and interpretability [2][3]. However, its main limitation is that it assumes a linear relationship between the features and the log-odds of the outcome, which may not capture complex nonlinear patterns in aviation data.

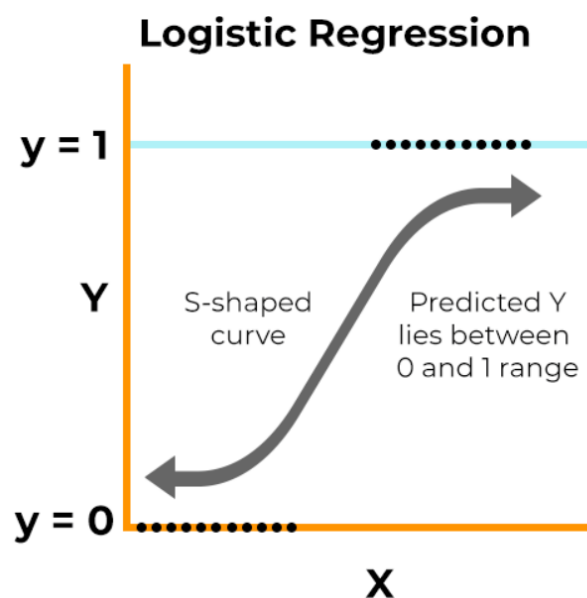


Figure 1: Logistic Regression Explanation Figure (Source: [7])

## 4.2 Decision Tree

Decision Tree is a non-linear supervised learning algorithm that splits the dataset into subsets based on feature values. It builds a tree-like structure where each internal node represents a decision rule, and each leaf node represents a classification outcome. Decision Trees are easy to interpret and can capture complex interactions between variables, making them suitable for flight delay prediction [2][4]. However, it is prone to overfitting, especially with large or noisy datasets.

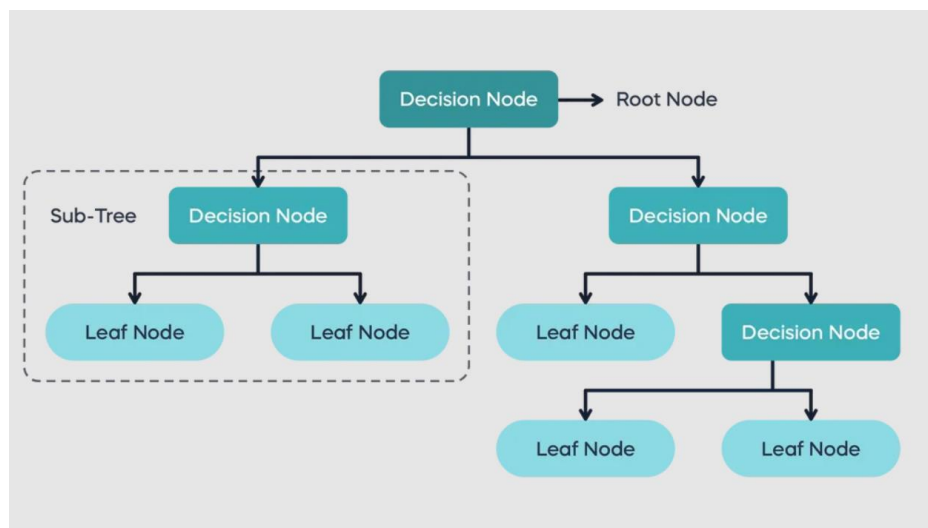


Figure 2: Decision Tree explanation Figure (Source: [8])

## 4.3 Random Forest

Random Forest is an ensemble learning method that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting. Each tree is trained on a random subset of the data, and the final prediction is obtained by majority voting. This approach increases robustness and is widely used in classification problems involving large and complex datasets such as flight data [2]. However, it is harder to understand how it makes decisions compared to a single decision tree, and it also needs more time and computing power to run.

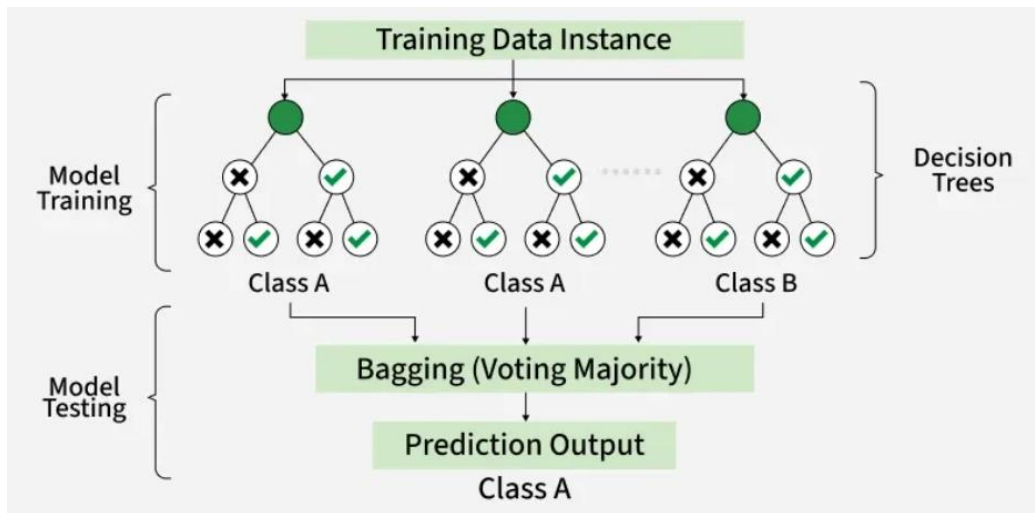


Figure 3: Random Forest Explanation Figure (Source: [\[9\]](#))

## 4.4 XGBoost

XGBoost is an advanced ensemble learning algorithm. It builds trees sequentially, where each new tree corrects the errors of the previous ones. XGBoost is known for its high performance, scalability, and ability to handle missing data. It is particularly effective in structured datasets and is often used in real-world predictive modeling tasks [\[3\]\[10\]](#). However, it requires careful parameter tuning and is more complex to implement.

## 4.5 Model Evaluation Metrics

Because of class imbalance, accuracy alone is not a sufficient evaluation metric. This project evaluates all models using the following metrics:

- **Accuracy:** the proportion of correctly classified instances out of all instances.
- **Precision:** the proportion of predicted delays that are actual delays.
- **Recall:** the proportion of actual delays that are correctly identified.
- **F1-Score:** the harmonic mean of precision and recall.

- **ROC-AUC:** is a score that shows how well a model separates two classes (like delayed flight vs not delayed). It measures how often the model ranks a correct answer higher than an incorrect one.

## **4.6 Handling Class Imbalance Using Class Weights**

Flight delay datasets are typically imbalanced, because on-time flights are more frequent than delayed ones. This will make the models biased toward the majority class and reduce the performance in detecting delays [\[2\]](#)[\[3\]](#). To address this issue, class weighting is applied during model training by assigning higher weights to the minority class and lower weights to the majority class. This helps the model focus more on correctly predicting delayed flights.

# **5. Literature Review**

## **5.1 Overview of Selected Studies**

### **5.1.1 Airline Flight Delay Prediction Using Machine Learning Models.**

Flight delays are gradually increasing and bring more financial difficulties and customer dissatisfaction to airline companies. To resolve this situation, supervised machine learning models were implemented to predict whether a flight will be delayed using historical flight data from JFK Airport. The main objective of the study is to compare multiple supervised classification models and identify the most effective approach for delay prediction. Since the dataset is imbalanced, with far more non-delayed flights than delayed ones, this issue is handled by using weighted evaluation metrics to ensure fair performance measurement across classes. Seven algorithms are evaluated, including Logistic Regression, KNN, SVM, Decision Tree, Random Forest, and Gradient Boosting,

using accuracy, precision, recall, and F1-score. The key findings show that tree-based models, particularly the Decision Tree, achieve the best performance with the highest accuracy (0.9778), while KNN performs the worst. Overall, the study concludes that ensemble and tree-based methods are more reliable for predicting flight delays compared to simpler classification models [\[2\]](#).

### **5.1.2 Predictive Modeling of Flight Delays at an Airport Using Machine Learning Methods.**

This study addresses the problem of predicting flight delays exceeding 15 minutes at a Turkish airport, which negatively affect airline efficiency and passenger satisfaction. The authors propose a machine learning-based approach using a dataset of over 40,000 flights collected over three years, including both operational and weather-related features. After preprocessing the data using one-hot encoding and handling class imbalance with SMOTE, several models were developed and compared, including Logistic Regression, Naïve Bayes, ANN, Random Forest, XGBoost, CatBoost, and LightGBM. The main objective was to evaluate different machine learning techniques for binary flight delay prediction and examine the impact of weather data and data balancing methods. The results show that XGBoost, CatBoost, and LightGBM achieved the best performance, with around 80% accuracy, while XGBoost provided the highest recall. SMOTE improved recall in some cases but did not consistently enhance overall performance, and weather features had limited influence compared to operational variables. Feature analysis using SHAP indicated that scheduled time, passenger number, distance, and weather conditions were the most important predictors of delays [\[3\]](#).

### **5.1.3 Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy.**

In this study published in *PLOS One* (2025), the authors examined how different machine learning algorithms can be used to predict flight delays, with a strong

focus on handling class imbalance in the dataset. They worked with a large Kaggle dataset containing more than 4 million flight records collected between 2018 and 2022. Delays were grouped into three categories: small (less than 15 minutes), medium (15–45 minutes), and large (45 minutes or more). The researchers applied several preprocessing steps, including feature engineering, selecting the top 25 most important features, MinMax scaling, hyperparameter tuning, and stratified 10-fold cross-validation. Since flight delay data is typically imbalanced, they also used three resampling techniques Random Oversampling (ROS), SMOTE, and ADASYN to improve model performance. The models tested were Decision Tree, Random Forest, Support Vector Classifier (SVC), Logistic Regression, K-Nearest Neighbors (KNN), and Naïve Bayes. Their performance was evaluated using Accuracy, F1-score, MCC, and ROC-AUC [\[4\]](#).

The results showed that Random Forest combined with Random Oversampling, and Decision Tree combined with SMOTE, performed the best, both achieving 90% accuracy and an F1-score of 0.91. SVC and Naïve Bayes also produced strong results, while KNN had the weakest performance among the tested models. Statistical tests, including the paired t-test and McNemar test, confirmed that the differences in performance were statistically significant. Overall, the study highlights the importance of addressing class imbalance and applying proper feature engineering techniques, and it shows that ensemble methods like Random Forest are especially effective for multiclass flight delay prediction.

#### **5.1.4 Integrating multiple data sources for improved flight delay prediction using explainable machine learning.**

In this study published in *Research in Transportation Business & Management* (2024), the authors developed a comprehensive data science framework to predict and analyze flight delays at Santiago Airport (SCL) in Chile. The research focuses on the integration of heterogeneous data sources, including operational flight

data, airport-specific attributes, geographic information, and real-time weather data from both origin and destination locations. To address the "black-box" nature of traditional predictive models, the researchers implemented an Explainable Artificial Intelligence (xAI) methodology. The preprocessing pipeline involved data fusion from multiple sources, feature selection, and the training of several machine learning algorithms, including Logistic Regression, Linear Discriminant Analysis (LDA), Decision Trees, Random Forest, and Support Vector Machines (SVM).

The results identified Linear Discriminant Analysis (LDA) as the most effective model for this specific dataset, providing a robust balance between predictive accuracy and computational efficiency. To gain deeper insights into the causes of disruptions, the study applied SHAP (SHapley Additive exPlanations) and Sobol Sensitivity Analysis to quantify feature importance. The analysis revealed that the most critical factors influencing flight delays were the international flight status, average temperature at the destination airport, local wind speed, and the ambient temperature at the departure airport. Overall, the study highlights that combining multi-source data with explainable AI techniques not only improves prediction performance but also provides actionable insights for airlines and airport operators to optimize scheduling and resource management [\[5\]](#).

### **5.1.5 A hybrid machine learning-based model for predicting flight delay through aviation big data.**

In this study published in Scientific Reports (2024), the author proposes a novel hybrid methodology to address the challenges of large-scale flight delay prediction using aviation big data. The research utilizes a dataset from JFK International Airport (2023) and introduces a sophisticated pipeline that combines advanced feature selection, clustering, and ensemble optimization. The preprocessing stage employs a combination of ANOVA for initial feature

importance analysis and the Forward Sequential Feature Selection (FSFS) algorithm to identify the most influential indicators. To manage the high dimensionality and volume of aviation data, the study implements an enhanced DBSCAN clustering strategy, which partitions the dataset into similar groups to decompose the prediction task into smaller, more manageable sub-problems.

The core of the predictive engine is a Coyote Optimization Algorithm-based Weighted Random Forest (COWRF) model. This novel structure optimizes individual decision trees using the Coyote Optimization Algorithm (COA) to tune split points and assigns weighted values to each tree component to enhance voting accuracy. The results demonstrate that the proposed hybrid approach achieves an average accuracy of 97.2%, representing a significant improvement over baseline ensemble methods and previous deep learning architectures like CNN-LSTM. Specifically, the integration of clustering reduced processing time by 39.17%, while the COA-driven optimization increased model accuracy by 5.3% compared to conventional Random Forest models. Statistical validations and cross-validation results confirm that the proposed framework is highly effective for real-time, large-scale aviation scheduling and planning applications [6].

## 5.2 Comparative Summary Table

*Table 1: Literature Review Comparative Table*

| Ref.                | Year | Dataset, Country    | Models Used                          | Best Result            | Imbalance Handling  |
|---------------------|------|---------------------|--------------------------------------|------------------------|---------------------|
| <a href="#">[2]</a> | 2021 | JFK Airport,<br>USA | LR, KNN, GNB,<br>DT, SVM, RF,<br>GBT | DT: 97.78%<br>accuracy | Weighted<br>metrics |

|                     |      |                                       |                                                       |                                                    |                                      |
|---------------------|------|---------------------------------------|-------------------------------------------------------|----------------------------------------------------|--------------------------------------|
| <a href="#">[3]</a> | 2024 | Turkish Airport,<br>Turkey            | LR, NB, ANN,<br>RF, XGBoost,<br>CatBoost,<br>LightGBM | XGBoost:<br>~80%<br>accuracy,<br>highest<br>recall | SMOTE                                |
| <a href="#">[4]</a> | 2025 | Kaggle Dataset,<br>USA<br>(2018-2022) | DT, RF, SVC,<br>LR,<br>KNN, NB                        | RF + ROS:<br>90%<br>accuracy                       | ROS, SMOTE,<br>ADASYN                |
| <a href="#">[5]</a> | 2024 | Santiago Airport<br>(SCL),<br>Chile   | LR, LDA, DT,<br>RF, SVM                               | LDA: best<br>accuracy                              | Feature<br>Selection+SHAP            |
| <a href="#">[6]</a> | 2024 | JFK Airport,<br>USA (2023)            | COWRF<br>(hybrid:<br>DBSCAN +<br>COA + RF             | COWRF:<br>97.2%<br>accuracy                        | ANOVA +<br>FSFS<br>Feature selection |

## 6. Dataset description

This project uses two datasets to introduce the analysis and model training.

First Dataset is **Flight Delay Dataset-2024**, This dataset contains flight delay and performance information for domestic flights in 2024. This dataset is combined from monthly merged from monthly BTS TranStats files in 2024 into a single cleaned dataset. Moreover, this dataset includes over 7 million rows and 35 columns, including features such as scheduled and actual flight times, delays, cancellations, diversions, and distances between airports. Furthermore, the

dataset was collected from BTS TranStats On-Time Performance database, and a monthly CSV files covering the period from January to December 2024 were merged into a single dataset using pandas, with cleaning steps, after merging the files only the most important 35 columns were kept.

Preprocessing steps were applied to prepare the data for analysis such as:

- The columns were renamed to snake\_case style, such as dest\_city\_name and dep\_delay.
- The flight\_date was converted into ISO format (YYYY-MM-DD).
- The cancelled and diverted columns were changed into binary values (0 and 1) to make them easier to use in analysis and machine learning models.
- Missing values in delay-related columns (carrier\_delay, weather\_delay, nas\_delay, security\_delay, late\_aircraft\_delay) were filled with 0, while preserving all other values as in the original data.

Second dataset is the **Airports Dataset**, this dataset was collected from a repository on github [\[11\]](#), this dataset provides structured information about global airports, including ITIA code, icao code, and coordinates. This dataset was merged with the flight delay dataset to enrich the analysis with airports attributes.

## 7. Data source explanation

The datasets used in this project were collected from two public sources. The flight delay dataset was downloaded from BTS TranStats On-Time Performance database, which contains detailed flight performance and delay information for domestic flights in 2024. The second dataset was collected from the airportsdata

Github repository [\[11\]](#), this dataset includes information about airports such as, longitude, latitude, ITIA code, and icao code.

Both datasets were used together to support the flight delay analysis and prediction.

## 8. Data cleaning and preprocessing

First of all, required libraries were imported at the begin of the notebook.

```
import pandas as pd
import numpy as np
import os
```

*Figure 4: imported libraries*

Here we read the dataset

```
df= pd.read_csv('data/raw/flight_data_2024.csv')

C:\Users\user\AppData\Local\Temp\ipykernel_39392\1977606443.py:1: DtypeWarning: Columns (0: cancellation_code) have mixed types. Specify dtype option
on import or set low_memory=False.
df= pd.read_csv('data/raw/flight_data_2024.csv')
```

We kept only the non-cancelled and non-diverted flights from the dataset because cancelled and diverted flights don't have an arrival delay value at all. Therefore, we can't classify them as delayed or not delayed.

```
[6]: df = df[df["cancelled"] == 0]
df = df[df["diverted"] == 0]
```

### 8.1 Sampling data

We created the target column which is weather the flight is delayed or not (0 = not delayed / 1 = delayed).

The flights that have an arrival delay equal to or larger than 15 minutes are flagged as delayed while rest are non-delayed.

This 15 minutes threshold follows the standard aviation reporting practice used in the aviation industry [\[12\]](#).

```
df['delayed']=(df['arr_delay']>=15).astype(int)
```

We sampled the data from approximately 7 million entries down to 500K entries, while preserving the original distribution of delayed and non-delayed flights to stay the same to ensure the class balance remained consistent.

```
print(f"Before a sampling, ratio of delayed flights {df['delayed'].mean()} and ratio of non delayed {(df['delayed']==0).mean()}")

total=500000
delayed_ratio= df['delayed'].mean()

number_delayed= int(total*delayed_ratio)
number_non_delayed= total-number_delayed

delayed_flights=df[df['delayed']==1].sample(n=number_delayed,random_state=7)
non_delayed_flights=df[df['delayed']==0].sample(n=number_non_delayed,random_state=7)

sampled_flights=pd.concat([delayed_flights,non_delayed_flights]).sample(frac=1, random_state=4)
print(f"After a stratified sampling, ratio of delayed flights {number_delayed/total} and ratio of non delayed {number_non_delayed/total}")
```

Before a sampling, the dataset contained of approximately 0.2082 delayed flights and 0.7918 non-delayed flights. After applying stratified sampling, the class remained unchanged with 0.2081 delayed flights and 0.7918 non-delayed, indicating that the original imbalance was maintained in the sampled dataset. Finally, the sampled dataset was saved locally within the project directory.

Write the sampled data to a new file

```
[9]: os.makedirs('data/sampled', exist_ok=True)
```

```
[10]: sampled_flights.to_csv('data/sampled/sampled_flights.csv')
```

## 8.2 Merging Airport Coordinate Data

Then we read the sampled dataset again and chose the important columns for our analysis and prediction.

```
[11]: df = pd.read_csv('data/sampled/sampled_flights.csv')

[12]: df=df[['year', 'month', 'day_of_month', 'day_of_week',
            'fl_date', 'distance',
            'origin', 'origin_city_name', 'origin_state_nm', 'dest',
            'dest_city_name', 'dest_state_nm', 'op_unique_carrier', 'op_carrier_fl_num',
            'crs_dep_time', 'crs_arr_time', 'delayed']]
```

Why we chose these columns,

- Year: captures yearly trends in delays.
- Month: captures seasonal effects (weather, holidays)

- `day_of_month`: captures daily patterns within a month
- `day_of_week`: captures weekend delay behaviour.
- `fl_date`: exact flight date.
- Distance: longer flights may have a higher probability of being delayed.
- Origin: departure airport code.
- `origin_city_name`: name of the city of the origin airport.
- `origin_state_nm`: origin state name.
- Dest: destination airport code.
- `dest_city_name`: name of the city of the destination airport.
- `dest_state_nm`: origin state name.
- `op_unique_carrier`: a unique airline operating the flight, represented by a code instead of the full airline name. Different airlines contain different delay patterns.
- `op_carrier_fl_num`: a flight number that is assigned to the airline operating the flight. This feature can help us find specific flight delay patterns.
- `crs_dep_time`: scheduled departure time, helps us capture rush hours.
- `crs_arr_time`: scheduled arrival time, helps capture airport traffic patterns.
- Delayed: target variable (0= not delayed, 1= delayed)

Then we wanted to introduce the geographic information (latitude and longitude) in the analysis of travels, so we got airports data and got the columns, `iata`, `lat`, and `lon`.

---

I fetched the airport data long and lat inorder to get the latitude and longitudde fro the origin and dest airports.

```
airports = pd.read_csv("data/raw/airports.csv")
airport_cord = airports[["iata", "lat", "lon"]]
```

---

srop the rows that contains na values

We dropped the rows that contains null values to ensure data quality.

```
airport_cord = airport_cord.dropna()
```

Also, we verified the correctness of the latitude and longitude.

```
[25]: airport_cord[
        (airport_cord['lat'] < -90) | (airport_cord['lat'] > 90) |
        (airport_cord['lon'] < -180) | (airport_cord['lon'] > 180)
    ]
```

```
[25]: iata lat lon
```

Then we renamed the columns of the airport\_cord (airports dataset) to match the columns in df (the flight delay dataset). Then we merged the airports dataset with df (the flight delay dataset) using the origin (ITIA code) as the key, keeping all rows from df (flight delay dataset) and adding the corresponding coordinates when available.

```
[16]: airport_cord.columns = ["origin", "origin_lat", "origin_lng"]
df=pd.merge(df,airport_cord,on='origin', how='left')
```

The same process was then applied to the destination airports.

```
airport_cord.columns = ["dest", "dest_lat", "dest_lng"]
df=pd.merge(df,airport_cord,on='dest', how='left')
```

At the end, we checked whether there are missing values after the merging process.

```
: print(df['origin_lat'].isna().any())
print(df['origin_lng'].isna().any())
print(df['dest_lat'].isna().any())
print(df['dest_lng'].isna().any())
```

```
False
False
False
False
```

```
no missing values in origin_lat
no missing values in origin_lng
```

```
no missing values in dest_lat
no missing values in dest_lng
```

```
Every origin and dest in our dataframe matched with the airport_cord dataframe
```

And as we see below, the latitude and longitude columns were merged.

```
df.head(3)
```

|   | origin_state_nm | dest | ... | dest_state_nm | op_unique_carrier | op_carrier_fl_num | crs_dep_time | crs_arr_time | delayed | origin_lat | origin_lng  | dest_lat  | dest_lng    |
|---|-----------------|------|-----|---------------|-------------------|-------------------|--------------|--------------|---------|------------|-------------|-----------|-------------|
| N | Tennessee       | MKE  | ... | Wisconsin     | WN                | 1404.0            | 1230         | 1405         | 0       | 36.124475  | -86.678181  | 42.946932 | -87.897064  |
| A | California      | JFK  | ... | New York      | DL                | 951.0             | 1100         | 1957         | 0       | 33.942496  | -118.408049 | 40.639928 | -73.778692  |
| A | California      | BUR  | ... | California    | WN                | 2485.0            | 1035         | 1145         | 0       | 37.362995  | -121.928621 | 34.200694 | -118.358667 |

## 8.3 Validate datatypes

As you see in the figure below, `fl_date` column is string while we need it as a datetime.

## Preprocessing and Cleaning the data

```
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  1000000 non-null  int64
1   month                 1000000 non-null  int64
2   day_of_month          1000000 non-null  int64
3   day_of_week           1000000 non-null  int64
4   fl_date                1000000 non-null  str
5   distance              1000000 non-null  float64
6   origin                 1000000 non-null  str
7   origin_city_name       1000000 non-null  str
8   origin_state_nm        1000000 non-null  str
9   dest                   1000000 non-null  str
10  dest_city_name         1000000 non-null  str
11  dest_state_nm          1000000 non-null  str
12  op_unique_carrier       1000000 non-null  str
13  op_carrier_fl_num       1000000 non-null  float64
14  crs_dep_time            1000000 non-null  int64
15  crs_arr_time            1000000 non-null  int64
16  delayed                1000000 non-null  int64
17  origin_lat              1000000 non-null  float64
18  origin_lng              1000000 non-null  float64
19  dest_lat                1000000 non-null  float64
20  dest_lng                1000000 non-null  float64
dtypes: float64(6), int64(7), str(8)
memory usage: 160.2 MB
```

We converted the string date into datetime so we can easily extract month, day of the month, day of the week, or year for patterns and seasonality.

```
[62]: df['fl_date']
```

```
[62]: 0      2024-01-06
      1      2024-08-04
      2      2024-01-12
      3      2024-04-03
      4      2024-11-17
      ...
      999995    2024-11-10
      999996    2024-05-27
      999997    2024-12-12
      999998    2024-05-20
      999999    2024-07-24
      Name: fl_date, Length: 1000000, dtype: str
```

We converted the fl\_date column to datetime type

```
[65]: df['fl_date'] = pd.to_datetime(df['fl_date'])
      df
```

Ok as you see in the figure below, fl\_date is now in a datetime type.

```
[67]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  1000000 non-null  int64
1   month                 1000000 non-null  int64
2   day_of_month          1000000 non-null  int64
3   day_of_week           1000000 non-null  int64
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64
6   origin                1000000 non-null  str
7   origin_city_name      1000000 non-null  str
8   origin_state_nm       1000000 non-null  str
9   dest                  1000000 non-null  str
10  dest_city_name        1000000 non-null  str
11  dest_state_nm         1000000 non-null  str
12  op_unique_carrier      1000000 non-null  str
13  op_carrier_fl_num      1000000 non-null  float64
14  crs_dep_time           1000000 non-null  int64
15  crs_arr_time           1000000 non-null  int64
16  delayed                1000000 non-null  int64
17  origin_lat             1000000 non-null  float64
18  origin_lng             1000000 non-null  float64
19  dest_lat              1000000 non-null  float64
20  dest_lng              1000000 non-null  float64
dtypes: datetime64[us](1), float64(6), int64(7), str(7)
memory usage: 160.2 MB
```

Next, we converted op\_carrier\_fl\_num from float into string, because it represents a categorical flight identifier, there is no meaning of being numerical value we are not using it for a calculations.

First, we checked if this column has any null values. Then, since the values were stored as floats, we verified whether they contained any decimal parts by checking if the remainder after division by 1 was zero. After that, since we

found the remainder of all values is zero. Therefore, all of `op_carrier_fl_num` can be converted from float to integer format successfully, and finally to string format.

There is no meaning of the `op_carrier_fl_num` to be a float data, it is just a string id

```
[239]: df['op_carrier_fl_num'].isna().sum()
```

```
[239]: np.int64(0)
```

check the datatype of the feature

```
[241]: df['op_carrier_fl_num'].dtype
```

```
[241]: dtype('float64')
```

since the values were stored as floats,

we verified whether they contained any decimal parts by checking if the remainder after division by 1 was zero.

```
[243]: df['op_carrier_fl_num'].apply(lambda x: x % 1 != 0).sum()
```

```
[243]: np.int64(0)
```

since we found the remainder of all values is zero.

Therefore, all of `op_carrier_fl_num` can be converted from float to integer format successfully,

```
[245]: df['op_carrier_fl_num']=df['op_carrier_fl_num'].astype(int)
```

now the `op_carrier_fl_num` feature is converted finally to string format.

```
[247]: df['op_carrier_fl_num']=df['op_carrier_fl_num'].astype(str)
df
```

```
[247]: df['op_carrier_fl_num']=df['op_carrier_fl_num'].astype(str)
df
```

|    | origin_state_nm | dest | ... | dest_state_nm | op_unique_carrier | op_carrier_fl_num | crs_dep_time | crs_arr_time | delayed | origin_lat | origin_lng  | dest_lat  | dest_lng    |
|----|-----------------|------|-----|---------------|-------------------|-------------------|--------------|--------------|---------|------------|-------------|-----------|-------------|
| N  | Tennessee       | MKE  | ... | Wisconsin     | WN                | 1404              | 1230         | 1405         | 0       | 36.124475  | -86.678181  | 42.946932 | -87.897064  |
| A  | California      | JFK  | ... | New York      | DL                | 951               | 1100         | 1957         | 0       | 33.942496  | -118.408049 | 40.639928 | -73.778692  |
| A  | California      | BUR  | ... | California    | WN                | 2485              | 1035         | 1145         | 0       | 37.362995  | -121.928621 | 34.200694 | -118.358667 |
| IY | New York        | LAS  | ... | Nevada        | B6                | 411               | 605          | 830          | 0       | 40.639928  | -73.778692  | 36.080343 | -115.152449 |
| H  | Ohio            | LAS  | ... | Nevada        | WN                | 3473              | 1245         | 1420         | 0       | 41.409407  | -81.854691  | 36.080343 | -115.152449 |

As you see in the figure below, `op_carrier_fl_num` column contains a string now.

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  1000000 non-null  int64
1   month                 1000000 non-null  int64
2   day_of_month          1000000 non-null  int64
3   day_of_week           1000000 non-null  int64
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64
6   origin                1000000 non-null  str
7   origin_city_name      1000000 non-null  str
8   origin_state_nm       1000000 non-null  str
9   dest                  1000000 non-null  str
10  dest_city_name        1000000 non-null  str
11  dest_state_nm         1000000 non-null  str
12  op_unique_carrier     1000000 non-null  str
13  op_carrier_fl_num     1000000 non-null  str
14  crs_dep_time          1000000 non-null  int64
15  crs_arr_time          1000000 non-null  int64
16  delayed               1000000 non-null  int64
17  origin_lat            1000000 non-null  float64
18  origin_lng            1000000 non-null  float64
19  dest_lat              1000000 non-null  float64
20  dest_lng              1000000 non-null  float64
dtypes: datetime64[us](1), float64(5), int64(7), str(8)
memory usage: 160.2 MB
```

## 8.4 Solving Inconsistency

Here we checked whether the string columns have additional spaces.

Here we are checking the string columns, if they have an entry contains additional space (not trimmed string)

```
!]: str_cols = ['origin', 'origin_city_name', 'origin_state_nm',
               'dest', 'dest_city_name', 'dest_state_nm',
               'op_unique_carrier', 'op_carrier_fl_num']

for c in str_cols:
    raw = df[c].unique()
    non_trimmed_strings = df[c].str.strip().str.upper().unique()
    if raw == non_trimmed_strings:
        print(f"no issues at {c} ({len(raw)} unique values)")
    else:
        print(f"{c}: {raw - non_trimmed_strings} values are not trimmed")

no issues at origin (348 unique values)
no issues at origin_city_name (342 unique values)
no issues at origin_state_nm (52 unique values)
no issues at dest (348 unique values)
no issues at dest_city_name (342 unique values)
no issues at dest_state_nm (52 unique values)
no issues at op_unique_carrier (15 unique values)
no issues at op_carrier_fl_num (6725 unique values)
```

Here we checked whether the string columns have inconsistent letter casing (case issues).

Here we are checking the string columns, if they have an entry is repeated with other cases (such as: BNA -> bna, Bna)

```
str_cols = ['origin', 'origin_city_name', 'origin_state_nm',
            'dest', 'dest_city_name', 'dest_state_nm',
            'op_unique_carrier', 'op_carrier_fl_num']
for c in str_cols:
    raw = df[c].nunique()
    capital_strings = df[c].str.upper().nunique() #this convert the string to capital then counts unique values
    if raw == capital_strings:
        print(f"No case problems at {c}, and ({raw} unique values)")
    else:
        print(f"There is a case problem at {c}, and ({raw-capital_strings} have problems)")
```

No case problems at origin, and (348 unique values)  
 No case problems at origin\_city\_name, and (342 unique values)  
 No case problems at origin\_state\_nm, and (52 unique values)  
 No case problems at dest, and (348 unique values)  
 No case problems at dest\_city\_name, and (342 unique values)  
 No case problems at dest\_state\_nm, and (52 unique values)  
 No case problems at op\_unique\_carrier, and (15 unique values)  
 No case problems at op\_carrier\_fl\_num, and (6725 unique values)

Here we checked whether the stored day of the month matches the actual day extracted from the flate date.

```
(df["day_of_month"] != df["fl_date"].dt.day).any()
```

np.False\_

Here we found out that days of the week are mapped from 1 to 7.

```
df[df["day_of_week"] != df["fl_date"].dt.dayofweek]["day_of_week"].value_counts()
```

```
day_of_week
1    150977
5    149603
7    147383
4    147093
2    138160
3    137664
6    129120
Name: count, dtype: int64
```

And because in python uses 0=Monday and our dataset is using 1=Monday. Then, we checked the consistency of the days of the week after adjusting them according to indexing differences.

```
(df["day_of_week"]-1 != df["fl_date"].dt.dayofweek).any()
```

np.False\_

Here we ensured that the month column matches the month from the flight date.

```
(df["month"] != df["fl_date"].dt.month).any()
```

np.False\_

Here we ensured that the year column matches the year from the flight date.

```
(df["year"] != df["fl_date"].dt.year).any()  
  
np.False_
```

Here we detected if there are invalid departure or arrival times outside valid HHMM range.

```
#check if there is zero or negative distances  
df[df['distance']<=0]
```

| year | month | day_of_month | day_of_week | fl_date | distance | origin | origin_city_name | origin_state_nm | dest | ... | dest_state_nm | op_unique_carrier | op_carrier_fl_nun |
|------|-------|--------------|-------------|---------|----------|--------|------------------|-----------------|------|-----|---------------|-------------------|-------------------|
|------|-------|--------------|-------------|---------|----------|--------|------------------|-----------------|------|-----|---------------|-------------------|-------------------|

0 rows × 21 columns

## 8.5 Duplicates

As we can see below, there is no duplicate data in the dataset.

```
df[df.duplicated()]
```

| year | month | day_of_month | day_of_week | fl_date | distance | origin | origin_city_name | origin_state_nm | dest | ... | dest_state_nm | op_unique_carrier | op_carrier_fl_nun |
|------|-------|--------------|-------------|---------|----------|--------|------------------|-----------------|------|-----|---------------|-------------------|-------------------|
|------|-------|--------------|-------------|---------|----------|--------|------------------|-----------------|------|-----|---------------|-------------------|-------------------|

0 rows × 21 columns

## 8.6 Missing Data

As shown below, there is no missing data in the dataframe.

## 4. Missing data

```
[36]: df.isna().sum()
```

```
[36]: year          0
      month         0
      day_of_month  0
      day_of_week   0
      fl_date        0
      distance       0
      origin         0
      origin_city_name 0
      origin_state_nm 0
      dest           0
      dest_city_name 0
      dest_state_nm  0
      op_unique_carrier 0
      op_carrier_fl_num 0
      crs_dep_time   0
      crs_arr_time   0
      delayed        0
      origin_lat     0
      origin_lng     0
      dest_lat       0
      dest_lng       0
      dtype: int64
```

As you see there are no Missing data in the dataframe

## 8.7 Outliers

Here we created a column called `distance_zscore` where we measured how many standard deviation each value is from the mean in standard deviations.

Then we created a column called `distance_outlier` that marks values as outlier if greater or equal to 3. As shown, you can see the result of number of outlier distance and not.

## 5. Outliers

```
df['distance_zscore'] = (df['distance'] - df['distance'].mean()) / df['distance'].std()
df['distance_outlier'] = df['distance_zscore'].apply(lambda x: abs(x) >= 3)
df["distance_outlier"].value_counts()
```

```
distance_outlier
False      991429
True        8571
Name: count, dtype: int64
```

Here we calculated the probability of flights where the distance is considered an outlier. Since the result was 0.86%, this indicates that only a small proportion of observations fall away from the z-score threshold.

```
(df["distance_outlier"]==True).sum()/len(df["distance_outlier"])  
  
np.float64(0.008571)
```

Only 0.86% of flights are distance outliers, that means the data is consistent.  
there exists some extremely short routes  
and exists some extremely long routes

## 8.8 Introduce features

We introduced a new column called “season”, it assigns a season label to each flight based on its month.

Add a season column for the flights  
it assigns a season label to each flight based on its month.

```
seasons = []  
  
for m in df['month']:  
    if m <= 2:  
        seasons.append('winter')  
    elif m <= 5:  
        seasons.append('spring')  
    elif m <= 8:  
        seasons.append('summer')  
    else:  
        seasons.append('fall')  
  
df['season'] = seasons
```

We introduced a new feature called “is\_weekend” which assigns a binary value to each flight indicating whether it occurs in weekend or not.

```
df['is_weekend'] = df['day_of_week'] >= 6
```

The feature type now is bool and i want to encode it into 0/1 (0=not weekend, 1=weekend).

```
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   year                  1000000 non-null  int64  
1   month                 1000000 non-null  int64  
2   day_of_month          1000000 non-null  int64  
3   day_of_week           1000000 non-null  int64  
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64 
6   origin                1000000 non-null  str     
7   origin_city_name      1000000 non-null  str     
8   origin_state_nm       1000000 non-null  str     
9   dest                 1000000 non-null  str     
10  dest_city_name        1000000 non-null  str     
11  dest_state_nm         1000000 non-null  str     
12  op_unique_carrier     1000000 non-null  str     
13  op_carrier_fl_num     1000000 non-null  str     
14  crs_dep_time          1000000 non-null  int64  
15  crs_arr_time          1000000 non-null  int64  
16  delayed               1000000 non-null  int64  
17  origin_lat            1000000 non-null  float64 
18  origin_lng            1000000 non-null  float64 
19  dest_lat              1000000 non-null  float64 
20  dest_lng              1000000 non-null  float64 
21  distance_zscore       1000000 non-null  float64 
22  distance_outlier      1000000 non-null  bool    
23  season                1000000 non-null  str     
24  is_weekend            1000000 non-null  bool    
dtypes: bool(2), datetime64[us](1), float64(6), int64(7), str(9)
memory usage: 177.4 MB
```

Here we converted the `is_weekend` feature into an integer type.

```
df['is_weekend'] = df['is_weekend'].astype(int)
```

As you see now it became an integer type.

```
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  1000000 non-null  int64
1   month                 1000000 non-null  int64
2   day_of_month          1000000 non-null  int64
3   day_of_week           1000000 non-null  int64
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64
6   origin                1000000 non-null  str
7   origin_city_name      1000000 non-null  str
8   origin_state_nm       1000000 non-null  str
9   dest                 1000000 non-null  str
10  dest_city_name        1000000 non-null  str
11  dest_state_nm         1000000 non-null  str
12  op_unique_carrier     1000000 non-null  str
13  op_carrier_fl_num     1000000 non-null  str
14  crs_dep_time          1000000 non-null  int64
15  crs_arr_time          1000000 non-null  int64
16  delayed               1000000 non-null  int64
17  origin_lat            1000000 non-null  float64
18  origin_lng            1000000 non-null  float64
19  dest_lat              1000000 non-null  float64
20  dest_lng              1000000 non-null  float64
21  distance_zscore       1000000 non-null  float64
22  distance_outlier      1000000 non-null  bool
23  season                1000000 non-null  str
24  is_weekend            1000000 non-null  int64
dtypes: bool(1), datetime64[us](1), float64(6), int64(8), str(9)
memory usage: 184.1 MB
```

Also, we converted the type of distance\_outlier from bool into integer, (1=is distance outlier, 0= not a distance outlier).

```
df['distance_outlier'] = df['distance_outlier'].astype(int)
```

As shown below, the datatype of distance\_outlier now is an integer.

```
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   year                  1000000 non-null  int64  
1   month                 1000000 non-null  int64  
2   day_of_month          1000000 non-null  int64  
3   day_of_week           1000000 non-null  int64  
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64
6   origin                1000000 non-null  str     
7   origin_city_name      1000000 non-null  str     
8   origin_state_nm       1000000 non-null  str     
9   dest                  1000000 non-null  str     
10  dest_city_name         1000000 non-null  str     
11  dest_state_nm          1000000 non-null  str     
12  op_unique_carrier      1000000 non-null  str     
13  op_carrier_fl_num      1000000 non-null  str     
14  crs_dep_time           1000000 non-null  int64  
15  crs_arr_time           1000000 non-null  int64  
16  delayed                1000000 non-null  int64  
17  origin_lat             1000000 non-null  float64
18  origin_lng             1000000 non-null  float64
19  dest_lat               1000000 non-null  float64
20  dest_lng               1000000 non-null  float64
21  distance_zscore        1000000 non-null  float64
22  distance_outlier       1000000 non-null  int64  
23  season                 1000000 non-null  str     
24  is_weekend             1000000 non-null  int64  
dtypes: datetime64[us](1), float64(6), int64(9), str(9)
memory usage: 190.7 MB
```

We first extracted the departure hour from the scheduled departure time (crs\_dep\_time).

Then, we created a new feature called period\_of\_day, which categorizes each flight as morning, afternoon, evening, or night based on its departure hour.

```
df['departure_hour'] = df['crs_dep_time'] // 100
```

```
result = []

for h in df['departure_hour']:
    if 5 <= h <= 11:
        result.append('morning')
    elif 12 <= h <= 16:
        result.append('afternoon')
    elif 17 <= h <= 20:
        result.append('evening')
    else:
        result.append('night')

df['period_of_day'] = result
```

As shown below, the *periodofday* feature is added successfully.

```
df.head(3)
```

| name  | origin_state_nm | dest | ... | origin_lat | origin_lng  | dest_lat  | dest_lng    | distance_zscore | distance_outlier | season | is_weekend | departure_hour | period_of_day |
|-------|-----------------|------|-----|------------|-------------|-----------|-------------|-----------------|------------------|--------|------------|----------------|---------------|
| 1, TN | Tennessee       | MKE  | ... | 36.124475  | -86.678181  | 42.946932 | -87.897064  | -0.601835       | 0                | winter | 1          | 12             | afternoon     |
| 5, CA | California      | JFK  | ... | 33.942496  | -118.408049 | 40.639928 | -73.778692  | 2.746703        | 0                | summer | 1          | 11             | morning       |
| 1, CA | California      | BUR  | ... | 37.362995  | -121.928621 | 34.200694 | -118.358667 | -0.901530       | 0                | winter | 0          | 10             | morning       |

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  1000000 non-null  int64
1   month                 1000000 non-null  int64
2   day_of_month          1000000 non-null  int64
3   day_of_week           1000000 non-null  int64
4   fl_date               1000000 non-null  datetime64[us]
5   distance              1000000 non-null  float64
6   origin                1000000 non-null  str
7   origin_city_name      1000000 non-null  str
8   origin_state_nm       1000000 non-null  str
9   dest                  1000000 non-null  str
10  dest_city_name         1000000 non-null  str
11  dest_state_nm          1000000 non-null  str
12  op_unique_carrier      1000000 non-null  str
13  op_carrier_fl_num      1000000 non-null  str
14  crs_dep_time           1000000 non-null  int64
15  crs_arr_time           1000000 non-null  int64
16  delayed                1000000 non-null  int64
17  origin_lat             1000000 non-null  float64
18  origin_lng             1000000 non-null  float64
19  dest_lat               1000000 non-null  float64
20  dest_lng               1000000 non-null  float64
21  distance_zscore        1000000 non-null  float64
22  distance_outlier       1000000 non-null  bool
23  season                 1000000 non-null  str
24  is_weekend             1000000 non-null  bool
25  departure_hour         1000000 non-null  int64
26  period_of_day          1000000 non-null  str
dtypes: bool(2), datetime64[us](1), float64(6), int64(8), str(10)
memory usage: 192.6 MB
```

Finally, the processed dataset was saved as a new file in the project directory.

```
df.to_csv('data/processed/processed_flights.csv')
```

## 9. Data summarization

### 9.1 Review Dataset after cleaning to do summarization on it

```
dataset.info()
✓ 0.5s

<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0             1000000 non-null   int64
1   year                   1000000 non-null   int64
2   month                  1000000 non-null   int64
3   day_of_month           1000000 non-null   int64
4   day_of_week            1000000 non-null   int64
5   fl_date                1000000 non-null   str
6   distance               1000000 non-null   float64
7   origin                 1000000 non-null   str
8   origin_city_name       1000000 non-null   str
9   origin_state_nm        1000000 non-null   str
10  dest                   1000000 non-null   str
11  dest_city_name         1000000 non-null   str
12  dest_state_nm          1000000 non-null   str
13  op_unique_carrier       1000000 non-null   str
14  op_carrier_fl_num       1000000 non-null   int64
15  crs_dep_time           1000000 non-null   int64
16  crs_arr_time           1000000 non-null   int64
17  delayed                1000000 non-null   int64
18  origin_lat             1000000 non-null   float64
19  origin_lng             1000000 non-null   float64
...
26  departure_hour         1000000 non-null   int64
27  period_of_day          1000000 non-null   str
dtypes: float64(6), int64(12), str(10)
```

Below are the columns of the cleaned dataset.

```
dataset.columns
✓ 0.0s

Index(['Unnamed: 0', 'year', 'month', 'day_of_month', 'day_of_week', 'fl_date',
      'distance', 'origin', 'origin_city_name', 'origin_state_nm', 'dest',
      'dest_city_name', 'dest_state_nm', 'op_unique_carrier',
      'op_carrier_fl_num', 'crs_dep_time', 'crs_arr_time', 'delayed',
      'origin_lat', 'origin_lng', 'dest_lat', 'dest_lng', 'distance_zscore',
      'distance_outlier', 'season', 'is_weekend', 'departure_hour',
      'period_of_day'],
      dtype='str')
```

The image below shows that the cleaned data has no null values.

```
dataset.isna().sum()
✓ 0.3s
```

|                   |   |
|-------------------|---|
| year              | 0 |
| month             | 0 |
| day_of_month      | 0 |
| day_of_week       | 0 |
| fl_date           | 0 |
| distance          | 0 |
| origin            | 0 |
| origin_city_name  | 0 |
| origin_state_nm   | 0 |
| dest              | 0 |
| dest_city_name    | 0 |
| dest_state_nm     | 0 |
| op_unique_carrier | 0 |
| op_carrier_fl_num | 0 |
| crs_dep_time      | 0 |
| crs_arr_time      | 0 |
| delayed           | 0 |
| origin_lat        | 0 |
| origin_lng        | 0 |
| dest_lat          | 0 |
| dest_lng          | 0 |
| distance_zscore   | 0 |
| distance_outlier  | 0 |
| season            | 0 |
| is_weekend        | 0 |
| departure_hour    | 0 |
| period_of_day     | 0 |
| dtype: int64      |   |

### 9.1.1 Print the number of origin and destination cities

The Image below shows that the number of origin and destination cities are 341 cities.

## Print the number of origin cities

```
origin_cities_count = dataset['origin_city_name'].nunique()  
print("Number of origin cities:", origin_cities_count)
```

✓ 0.0s

Number of origin cities: 341

## Print the number of destination cities

```
dest_cities_count= dataset['dest_city_name'].nunique()  
print("number of destination cities", dest_cities_count)
```

✓ 0.0s

number of destination cities 341

### 9.1.2 Drop unnamed column

We drop it because there are no need to it and to have a clear result for mode

```
dataset = dataset.iloc[:,1:]
```

We did a slicing to select all rows (:) and all columns starting from the second column (1:), which removes the first column from the dataframe. This is because the unwanted “unnamed” column is the first column.

### 9.1.3 Mode

The mode represents the most frequently occurring value in each column

- We can see that the largest number of flight trips in 2024 occurred during the morning period of the day, in July (month 7), and on the 15th day of the month. the reason may be that morning flights benefit from clearer air and more stable weather conditions compared to other times of the day. July also has a high

number of flights because it is part of the summer vacation season, when many people travel.

- We can also say that October 17th is the date with the highest number of flights.
- A distance of 337 is the most common flight distance in the dataset.
- Fall is the season with the highest number of flight trips. In addition, most flights do not occur on weekends, and the majority of flights have a duration of approximately 7 h.

```
dataset.mode()
```

Python

|   | year | month | day_of_month | day_of_week | fl_date    | distance | origin | origin_city_name | origin_state_nm | dest | ... | origin_lat | origin_lng | dest_lat | dest   |
|---|------|-------|--------------|-------------|------------|----------|--------|------------------|-----------------|------|-----|------------|------------|----------|--------|
| 0 | 2024 | 7     | 15           | 1           | 2024-10-17 | 337.0    | ATL    | Chicago, IL      | Texas           | ATL  | ... | 33.6367    | -84.427864 | 33.6367  | -84.42 |

1 rows x 27 columns

## 9.2 Categorical Data Analysis

### 9.2.1.1 Check which season is the most season that may have delayed

In this section we try to analysis categorical data such like the season, period of day , dest city name , month , day\_of\_ monthin this cell we try to check which season have the most delayed

we find that summer is the season that have the most delayed may that happen because people late at the flight , then we have spring , fall , winter

### 9.2.1.2find the percentage of delay for each season

for this cell we try to find the delayed percentage for each season to pass the issues that for example summer have a large number of trips so logically it may have large number of delayed flights.

Therefore, we find delayed percentage for each season based on the total number of flights and the total number of delayed flights and we find that summer remain them most season that have delayed , but we find a diff between fall and winter , we find that winter have more delayed based on the delayed percentage than fall.

```
dataset[dataset['delayed'] == 1]['season'].value_counts()
34] ✓ 0.1s

season
summer    68168
spring    56217
fall      53836
winter    29950
Name: count, dtype: int64

## find the persantage of delay for each season

dataset.groupby("season")["delayed"].mean().sort_values()
49] ✓ 0.1s

season
fall      0.160014
winter    0.200977
spring    0.221921
summer    0.260967
Name: delayed, dtype: float64
```

### 9.2.2 Which period of the day have the most delayed

in this cell we find that evening is the period of day that have the most delayed number of flights , then we can night afternoon last one is the moorning.

```
dataset[dataset['delayed'] == 1]['period_of_day'].value_counts()
```

38] ✓ 0.1s

```
.. period_of_day
afternoon    67666
evening      65055
morning      57363
night        18087
Name: count, dtype: int64
```

```
dataset.groupby("period_of_day")["delayed"].mean().sort_values()
```

51] ✓ 0.1s

```
.. period_of_day
morning      0.137376
afternoon    0.231360
night        0.268992
evening      0.292084
Name: delayed, dtype: float64
```

### 9.2.3 which city have the most delayed flights.

we can see that flight that go to Hobbs, NM is the most flights that have delayed.

```
dataset[dataset['delayed'] == 1]['dest_city_name'].value_counts()
```

43] ✓ 0.4s

```
.. dest_city_name
Chicago, IL          10515
Dallas/Fort Worth, TX 10470
Atlanta, GA          8410
New York, NY          8133
Denver, CO           8077
...
Moab, UT              1
New Bern/Morehead/Beaufort, NC 1
Pago Pago, TT         1
Dillingham, AK         1
Morgantown, WV         1
Name: count, Length: 338, dtype: int64
```

### 9.3.4 which month have the most delayed flights

we can see that flight that go at july (7) is the most flights that have delayed and this is may because of conjugation so people me late to arrive at airport.

```
dataset[dataset['delayed'] == 1]['month'].value_counts()
```

4] ✓ 0.0s

| month |       |
|-------|-------|
| 7     | 26093 |
| 5     | 22924 |
| 6     | 21729 |
| 8     | 20346 |
| 1     | 18299 |
| 12    | 17625 |
| 3     | 17524 |
| 4     | 15769 |
| 9     | 12814 |
| 11    | 11927 |
| 2     | 11651 |
| 10    | 11470 |

Name: count, dtype: int64

## 9.3 Numerical Data Analysis

### 9.3.1 Feature Correlation Analysis

for this interpretation I try to find the most strong and weak relationships

- very strong relationship between `departure_hour` and `crs_dep_time` (0.9993), which is expected since `departure_hour` was directly built from `crs_dep_time`.
- `crs_dep_time` and `crs_arr_time` share a moderate relationship (0.685), as flights departing later also tend to arrive later.
- strong relationship between `is_weekend` and `day_of_week` (0.784), consistent with how the feature was constructed.

- moderate relationship between origin\_lng and dest\_lng (0.604), and origin\_lat and dest\_lat (0.230), reflecting that many flights operate within similar geographic corridors.
- The distance and distance\_zscore columns are perfectly correlated (1.000) since the z-score is derived directly from distance.
- The distance\_outlier and distance\_zscore show a moderate positive relationship (0.344), as expected.
- The year column produces NaN values across all pairs due to zero variance (all records are from 2024).

dataset.corr(numeric\_only=True)

✓ 0.8s

|                   | year | month     | day_of_month | day_of_week | distance  | op_carrier_fl_num | crs_dep_time | crs_arr_time | delayed   | origin_lat | origin_lng | dest_lat  | dest_lng  | distance_zscore |
|-------------------|------|-----------|--------------|-------------|-----------|-------------------|--------------|--------------|-----------|------------|------------|-----------|-----------|-----------------|
| year              | NaN  | NaN       | NaN          | NaN         | NaN       | NaN               | NaN          | NaN          | NaN       | NaN        | NaN        | NaN       | NaN       | NaN             |
| month             | NaN  | 1.000000  | 0.003272     | 0.006589    | -0.004969 | 0.031510          | -0.003422    | -0.008256    | -0.038527 | 0.019580   | -0.004784  | 0.016910  | -0.006033 | -0.004969       |
| day_of_month      | NaN  | 0.003272  | 1.000000     | 0.012904    | 0.001442  | 0.003405          | 0.001653     | -0.000545    | 0.009072  | -0.000169  | 0.000835   | -0.001001 | -0.000486 | 0.001442        |
| day_of_week       | NaN  | 0.006589  | 0.012904     | 1.000000    | 0.008157  | -0.005467         | 0.003731     | -0.000665    | 0.027821  | -0.008472  | -0.002898  | -0.008459 | -0.002243 | 0.008157        |
| distance          | NaN  | -0.004969 | 0.001442     | 0.008157    | 1.000000  | -0.372124         | -0.008622    | 0.015483     | 0.016279  | -0.047730  | -0.115411  | -0.048448 | -0.116084 | 1.000000        |
| op_carrier_fl_num | NaN  | 0.031510  | 0.003405     | -0.005467   | -0.372124 | 1.000000          | 0.027906     | 0.017634     | -0.017495 | 0.093442   | 0.129872   | 0.098733  | 0.130769  | -0.372124       |
| crs_dep_time      | NaN  | -0.003422 | 0.001653     | 0.003731    | -0.008622 | 0.027906          | 1.000000     | 0.685170     | 0.161730  | -0.031345  | 0.000153   | 0.031230  | -0.007995 | -0.008622       |
| crs_arr_time      | NaN  | -0.008256 | -0.000545    | -0.000665   | 0.015483  | 0.017634          | 0.685170     | 1.000000     | 0.135774  | -0.013375  | 0.015071   | 0.017287  | 0.004897  | 0.015483        |
| delayed           | NaN  | -0.038527 | 0.009072     | 0.027821    | 0.016279  | -0.017495         | 0.161730     | 0.135774     | 1.000000  | -0.014733  | 0.027353   | -0.010078 | 0.015965  | 0.016279        |
| origin_lat        | NaN  | 0.019580  | -0.000169    | -0.008472   | -0.047730 | 0.093442          | -0.031345    | -0.013375    | -0.014733 | 1.000000   | 0.024407   | 0.229931  | -0.016001 | -0.047730       |
| origin_lng        | NaN  | -0.004784 | 0.000835     | -0.002898   | -0.115411 | 0.129872          | 0.000153     | 0.015071     | 0.027353  | 0.024407   | 1.000000   | -0.015425 | 0.604123  | -0.115411       |
| dest_lat          | NaN  | 0.016910  | -0.001001    | -0.008459   | -0.048448 | 0.098733          | 0.031230     | 0.017287     | -0.010078 | 0.229931   | -0.015425  | 1.000000  | 0.023779  | -0.048448       |
| dest_lng          | NaN  | -0.006033 | -0.000486    | -0.002243   | -0.116084 | 0.130769          | -0.007995    | 0.004897     | 0.015965  | -0.016001  | 0.604123   | 0.023779  | 1.000000  | -0.116084       |
| distance_zscore   | NaN  | -0.004969 | 0.001442     | 0.008157    | 1.000000  | -0.372124         | -0.008622    | 0.015483     | 0.016279  | -0.047730  | -0.115411  | -0.048448 | -0.116084 | 1.000000        |
| distance_outlier  | NaN  | -0.002339 | -0.001175    | 0.001031    | 0.344296  | -0.052020         | 0.015623     | -0.018303    | -0.001716 | -0.045395  | -0.143138  | -0.045079 | -0.141786 | 0.344296        |
| is_weekend        | NaN  | 0.012381  | 0.014172     | 0.784152    | 0.011612  | -0.006005         | -0.000825    | -0.002096    | 0.011146  | -0.011965  | -0.002195  | -0.011419 | -0.002203 | 0.011612        |
| departure_hour    | NaN  | -0.003318 | 0.001950     | 0.003677    | -0.008828 | 0.027571          | 0.999331     | 0.684257     | 0.161462  | -0.031945  | 0.000426   | 0.031416  | -0.008119 | -0.008828       |

Generate Code Markdown

### 9.3.2 Interpretation of describe Results

- All the dataset came from 2024 year and it's take the whole month and all days on the month and we can see that by the mean of the month (6.5) and day\_of\_month (15) and all the days on the week (3).
- The average flight distance is approximately 834 miles, with a minimum of 31 miles and a maximum of 5,095 miles, indicating a wide range of short-haul to long-haul domestic flights.
- Scheduled departure times (crs\_dep\_time) range from 0001 to 2359 with a mean around 1326 (early afternoon), while arrival times average around 1490.
- The target variable delayed has a mean of 0.208, confirming the class imbalance where approximately 20.8% of flights are delayed.

- Latitude values for origin airports range from  $-14.3$  to  $71.3$ , and longitude from  $-176.6$  to  $145.7$ , reflecting the geographic diversity of U.S. domestic airports including Alaska, Hawaii, and U.S. territories.

dataset.describe()

✓ 0.7s

|       | year      | month          | day_of_month   | day_of_week    | distance       | op_carrier_fl_num | crs_dep_time   | crs_arr_time   | delayed        | origin_lat     | origin_lng     | dest_          |
|-------|-----------|----------------|----------------|----------------|----------------|-------------------|----------------|----------------|----------------|----------------|----------------|----------------|
| count | 1000000.0 | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1000000.000000    | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1000000.000000 |
| mean  | 2024.0    | 6.597970       | 15.786884      | 3.983077       | 834.461610     | 2502.772380       | 1325.796484    | 1490.292645    | 0.208171       | 36.507990      | -94.832342     | 36.5967        |
| std   | 0.0       | 3.396064       | 8.787083       | 2.010310       | 597.276968     | 1652.043716       | 492.424283     | 518.898428     | 0.406000       | 5.997163       | 18.546931      | 8.9941         |
| min   | 2024.0    | 1.000000       | 1.000000       | 1.000000       | 31.000000      | 1.000000          | 1.000000       | 1.000000       | 0.000000       | -14.331682     | -176.642482    | -14.3316       |
| 25%   | 2024.0    | 4.000000       | 8.000000       | 2.000000       | 399.000000     | 1150.000000       | 906.000000     | 1103.000000    | 0.000000       | 32.898639      | -111.150260    | 32.8986        |
| 50%   | 2024.0    | 7.000000       | 16.000000      | 4.000000       | 880.000000     | 2224.000000       | 1320.000000    | 1515.000000    | 0.000000       | 36.894604      | -87.908150     | 36.8946        |
| 75%   | 2024.0    | 10.000000      | 23.000000      | 6.000000       | 1069.000000    | 3717.000000       | 1735.000000    | 1925.000000    | 0.000000       | 40.777242      | -80.951379     | 40.7772        |
| max   | 2024.0    | 12.000000      | 31.000000      | 7.000000       | 5095.000000    | 8819.000000       | 2359.000000    | 2359.000000    | 1.000000       | 71.284861      | 145.729986     | 71.2841        |

Python

### 9.3.3 Feature Correlation Analysis how every numeric column relates to delayed

We can see that most columns have a very weak linear relationship with delayed. However, crs\_dep\_time, crs\_arr\_time, and departure\_hour show a weak positive relationship with delayed, with correlation values of 0.161730, 0.135774, and 0.161462, respectively and NaN because we have only the year. 2024.

### Feature Correlation Analysis how every numeric column relates to delayed

dataset.corr(numeric\_only=True)["delayed"]

✓ 0.8s

|                   |           |
|-------------------|-----------|
| year              | NaN       |
| month             | -0.038527 |
| day_of_month      | 0.009072  |
| day_of_week       | 0.027621  |
| distance          | 0.016279  |
| op_carrier_fl_num | -0.017495 |
| crs_dep_time      | 0.161730  |
| crs_arr_time      | 0.135774  |
| delayed           | 1.000000  |
| origin_lat        | -0.014733 |
| origin_lng        | 0.027353  |
| dest_lat          | -0.010078 |
| dest_lng          | 0.015965  |
| distance_zscore   | 0.016279  |
| distance_outlier  | -0.001716 |
| is_weekend        | 0.011146  |
| departure_hour    | 0.161462  |

Name: delayed, dtype: float64

### 9.3.4 Calculate the delay percentage for each airline.

In the image below, we calculated the delay percentage in each airline,

This result gives us an insight about the time-on performance for each airline. The lowest delay percentage is in YX airline (14.240030). telling us that this airline has better time-on performance. While, the highest delay percentage is in F9 (28.801249), meaning that it has the worst time-on performance across all the airlines in the dataset.

```
calculate the delay percenatge in each airline

airline_delay=dataset.groupby('op_unique_carrier')['delayed'].mean()*100

print("Airline delay Pertcentge:")
print(airline_delay.sort_values(ascending=False))

✓ 0.0s

Airline delay Pertcentge:
op_unique_carrier
F9      28.801249
AA      25.917630
B6      25.576694
NK      24.256180
AS      22.441081
G4      22.363223
OH      21.916085
MQ      21.100917
WN      20.588818
UA      20.225472
OO      18.743481
DL      17.007848
9E      15.742603
HA      15.493456
YX      14.240030
Name: delayed, dtype: float64
```

### 9.3.5 Calculate the origin delay percentage.

As shown in the image below, we calculated the origin delay percentage in each airline,

Some airports has extremely high delay percentage like MGW (100%) this tells us that this origin has traffic-related issues and Extremely low delay percentage

like (SPI,CNY,PPG) has good time-on performance or maybe we don't have enough data about them inside the dataset.

```
calculate the airport (origin) delay percentage

origin_delay=dataset.groupby('origin')['delayed'].mean()*100

print("Origin delay Percentage")
print(origin_delay.sort_values(ascending=False))
```

✓ 0.0s

```
Origin delay Percentage
origin
MGW    100.000000
HYA     55.555556
HTS     52.631579
EWN     50.000000
EAU     50.000000
...
PIH      2.083333
PPG      0.000000
PQI      0.000000
CNY      0.000000
SPI      0.000000
Name: delayed, Length: 347, dtype: float64
```

Some airports has extremely high delay percentage like MGW (100%) this tells us that this origin has traffic-related issues and Extremely low delay percentage like (SPI,CNY,PPG) has good time-on performance or maybe it don't have enough for it inside the dataset

Acti

### 9.3.6 Calculate the number of flight observations in each origin.

In the image below, we calculated the number of flights records for each origin in the dataset.

the number of flights entries for MGW is one, this gives us an indicator that the delay percentage of MGW (100%) was because we don't have enough data about it in the dataset.

Also, the number of flights entries for CNY is one, this gives us an indicator that the delay percentage of CNY (0%) was because we don't have enough data about it in the dataset.

These findings highlights the importance of considering the number of records associated with each airport when interpreting delay rates. Because, airports with few number of flight observations may reflect a wrong patterns about the flight delays.

count the number of flights for each origin in the dataset

```
org_counts=df['origin'].value_counts()
print("Flights per Origin Airport:")
print(org_counts)
```

Flights per Origin Airport:

origin

ATL 24219

DFW 21940

DEN 21910

ORD 19925

CLT 15328

...

PQI 3

EAU 2

EWN 2

MGW 1

CNY 1

Name: count, Length: 347, dtype: int64

## 10. Visualizations and interpretation

### 10.1 Data Distribution

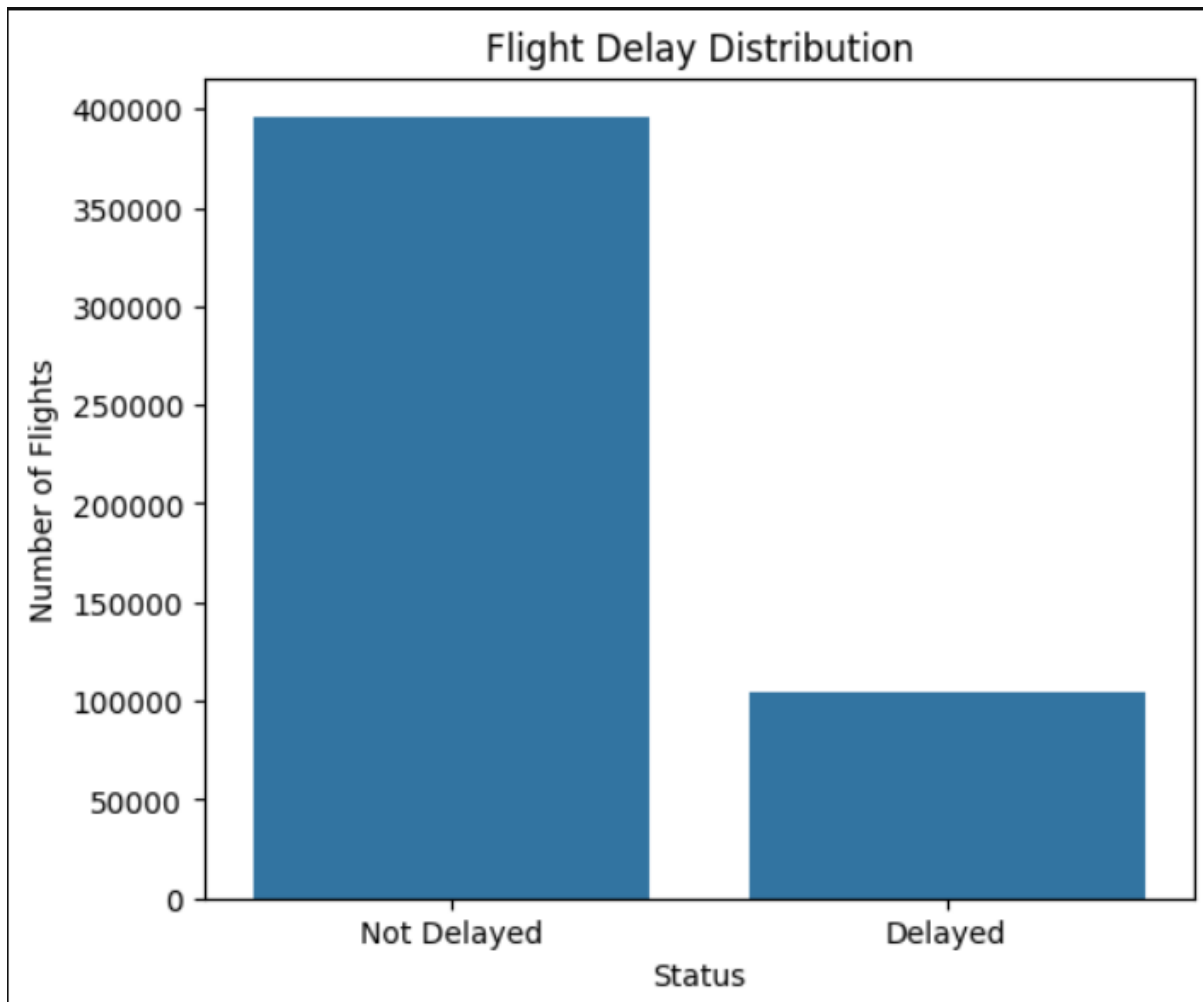
In this section we need to know the distribution for the delayed and non delayed flights, so we use the bar plot to make it easy to compare between the classes

```
delay_counts = df['delayed'].value_counts()

plt.figure(figsize=(6,5))
sns.barplot(
    x=['Not Delayed', 'Delayed'],
    y=delay_counts.values
)

plt.title('Flight Delay Distribution')
plt.ylabel('Number of Flights')
plt.xlabel('Status')

plt.show()
```



So depends on the chart above around 400K of data are not delayed flights and the delayed ones around 100K flight.

This means that the data is imbalance, but it shows the reality or real data in the world because not every flight can be delayed, to make it more accurate we want to get the ratio in the image below :

```
delay_ratio = df['delayed'].mean() * 100  
  
print(f"{delay_ratio:.2f}% of flights are delayed.")  
  
20.82% of flights are delayed.
```

So delayed flights are 20.82% of our dataset

The imbalance in data mean that we should use other evaluation metrics not just the accuracy maybe f1- score, precision and recall for the machine learning predict model.

## 10.2 Average Delayed Percentage by Month

We need it to see if there is a pattern for the delayed flights throughout the year

```
monthly_delay = df.groupby('month')['delayed'].mean() * 100

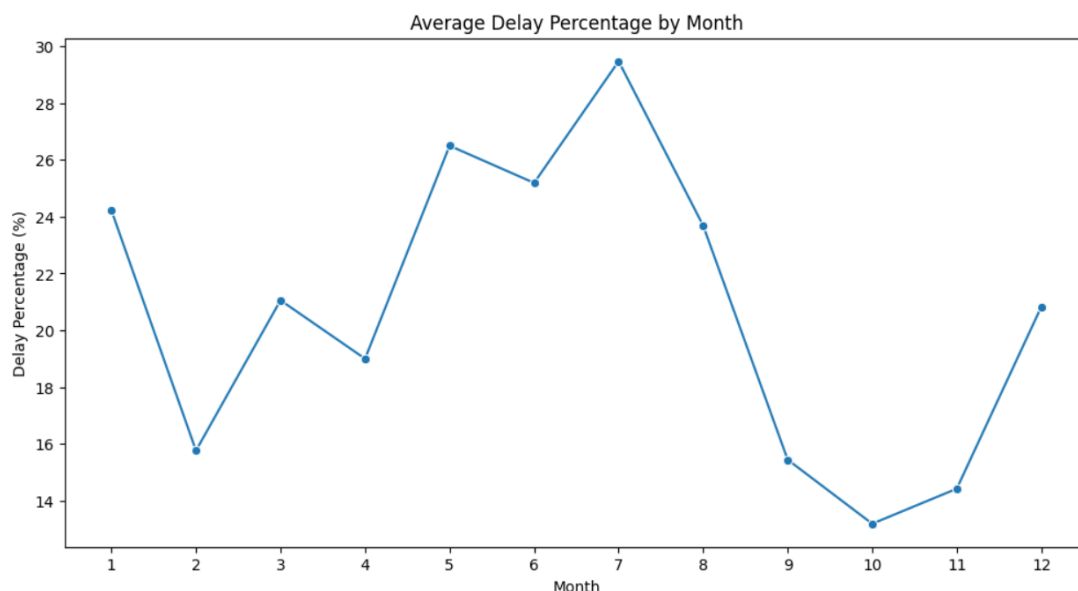
plt.figure(figsize=(12,6))

sns.lineplot(
    x=monthly_delay.index,
    y=monthly_delay.values,
    marker='o'
)

plt.title('Average Delay Percentage by Month')
plt.xlabel('Month')
plt.ylabel('Delay Percentage (%)')

plt.xticks(range(1,13))

plt.show()
```



We use the line chart to see the average delayed flights for each month throw the year.

The chart shows that the average delay is different throw the months some months like July has a highest delay percentage and on the other side October has the lowest delay percentage, that means the months and seasons affect the flights and make a delay sometimes.

```
monthly_delay.sort_values(ascending=False)

month
7      29.468599
5      26.495904
6      25.188256
1      24.219290
8      23.684877
3      21.056139
12     20.826740
4      19.000024
2      15.766499
9      15.444806
11     14.424363
10     13.191304
Name: delayed, dtype: float64
```

In the image above we sort the flights percentage descending, so we can see the (7,5,6) months are the most delayed flight percentage in them because of the seasons (Spring and Summer) and for the vacations for the employees.

And (11,10) are the lowest flight delay because it is the autumn season and rarely employees take their vacations in these months.

We can take insight that the delayed travels most likely related to the high traffic demand and weather cases as in January, that helps airlines to improve their plans and make decisions.

### 10.3 Average Delayed Percentage by Day of Week

Here we need to check if the delays happened in a frequently days in the week or there is no relation between them.

```
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

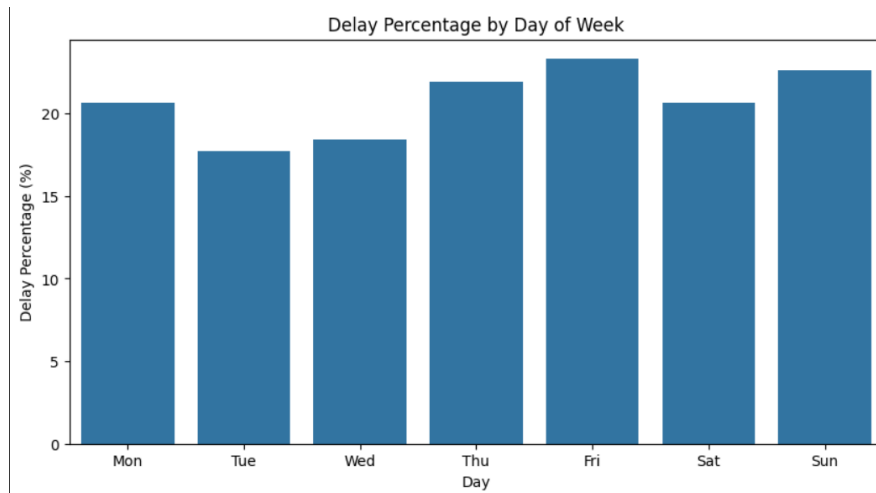
weekly_delay = df.groupby('day_of_week')['delayed'].mean() * 100

plt.figure(figsize=(10,5))

sns.barplot(
    x=days,
    y=weekly_delay.values
)

plt.title('Delay Percentage by Day of Week')
plt.xlabel('Day')
plt.ylabel('Delay Percentage (%)')

plt.show()
```



From the bar chart here we can find that most delayed flights in Friday and Sunday, the reason is because that they are the weekends of the week for employees, so it makes more traffic in the Airports and flights.

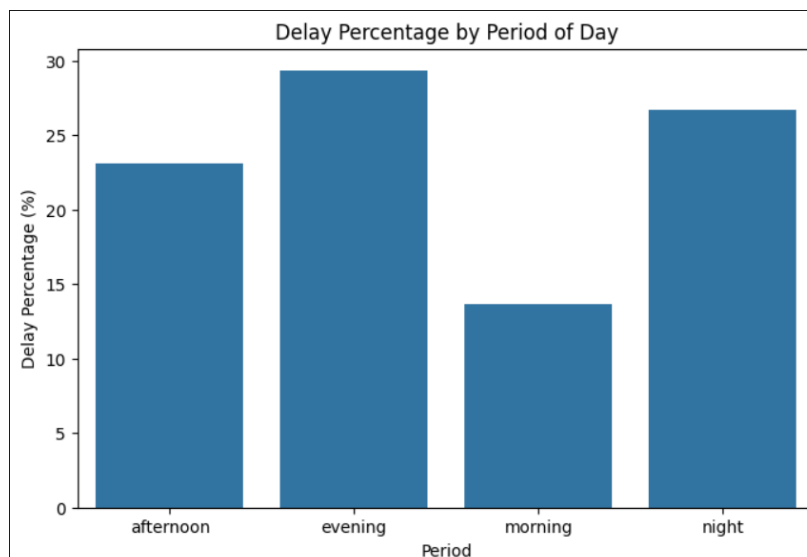
## 10.4 Average Delayed by Period of Day

Need to see how the time can affect flight delays

```
period_delay = df.groupby('period_of_day')['delayed'].mean() * 100
plt.figure(figsize=(8,5))

sns.barplot(
    x=period_delay.index,
    y=period_delay.values
)

plt.title('Delay Percentage by Period of Day')
plt.xlabel('Period')
plt.ylabel('Delay Percentage (%)')
plt.show()
```



As we see the period is classify by four classes and as we see (evening and night) the two periods that have the most delay time.

Also means that the flights schedule later in same day are more likely to be delayed comparing it with morning.

And in case any one flight delayed in some times it affect many flights after it.

And the early time flights if they delayed they affect the flights after that time, for example in case a flight in afternoon is delayed the flights that scheduled in evening and night also delayed.

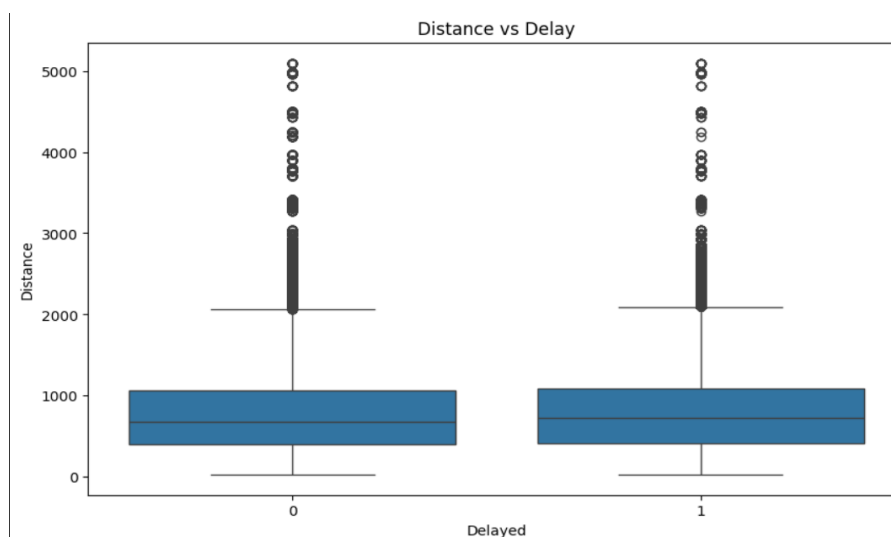
## 10.5 Distance vs Delay

Need it analyze if there is a relation between distance and delays

```
plt.figure(figsize=(10,6))

sns.boxplot(
    x='delayed',
    y='distance',
    data=df
)

plt.title('Distance vs Delay')
plt.xlabel('Delayed')
plt.ylabel('Distance')
plt.show()
```



The Boxplots shows that the distance don't have a big different between the Delayed and not delayed flights.

Both have similar median and interquartiles ranges.

The outliers here are the flights that have a long distance.

so from what we see the data is similar between the groups that means the distance field is not strong predictor for flight delays, so it can be useful when we combine it with other variables.

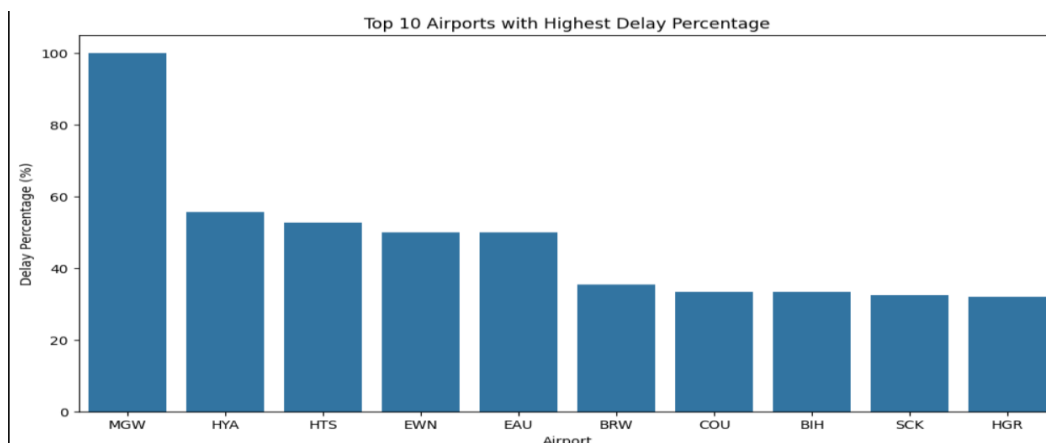
## 10.6 Top 10 Airports with Highest Delay Percentage

```
top_airports = (
    df.groupby('origin')['delayed']
      .mean()
      .sort_values(ascending=False)
      .head(10)
      * 100
)

plt.figure(figsize=(12,6))

sns.barplot(
    x=top_airports.index,
    y=top_airports.values
)

plt.title('Top 10 Airports with Highest Delay Percentage')
plt.xlabel('Airport')
plt.ylabel('Delay Percentage (%)')
plt.show()
```



These are the 10 airports have the highest delay percentage, and as we see there the MGW (Morgantown Municipal Airport) has around 100% chance to delay, after search we found that because of the weather in that airport and some restrictions related to its shorter runway length. [\[13\]](#)[\[14\]](#)

That make a congestion and cause a delays.

## 10.7 Top 10 Airlines with Highest Delay

Need to check the delay differences between the airlines

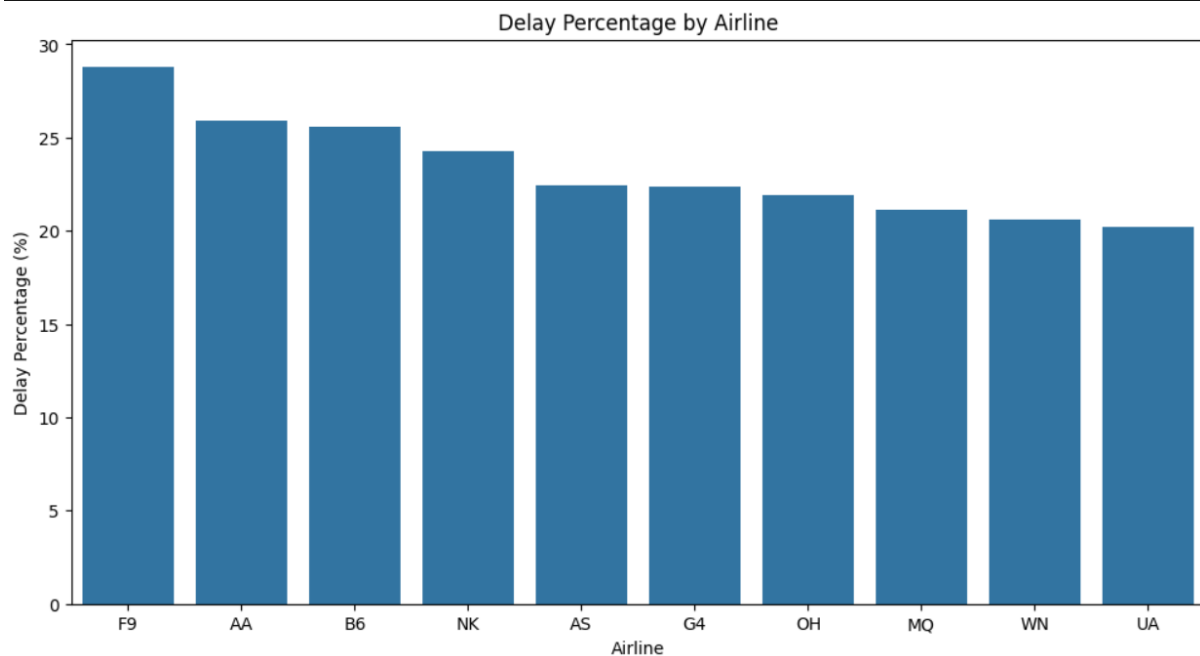
```
carrier_delay = (
    df.groupby('op_unique_carrier')['delayed']
    .mean()
    .sort_values(ascending=False)
    .head(10)
    * 100
)

plt.figure(figsize=(12,6))

sns.barplot(
    x=carrier_delay.index,
    y=carrier_delay.values
)

plt.title('Delay Percentage by Airline')
plt.xlabel('Airline')
plt.ylabel('Delay Percentage (%)')

plt.show()
```



## 10.8 Correlation Heatmap

To check the strength of relation between the numeric values

```

numeric_cols = [
    'month',
    'day_of_month',
    'day_of_week',
    'distance',
    'origin_lat',
    'origin_lng',
    'dest_lat',
    'dest_lng',
    'departure_hour',
    'delayed'
]

corr = df[numeric_cols].corr()

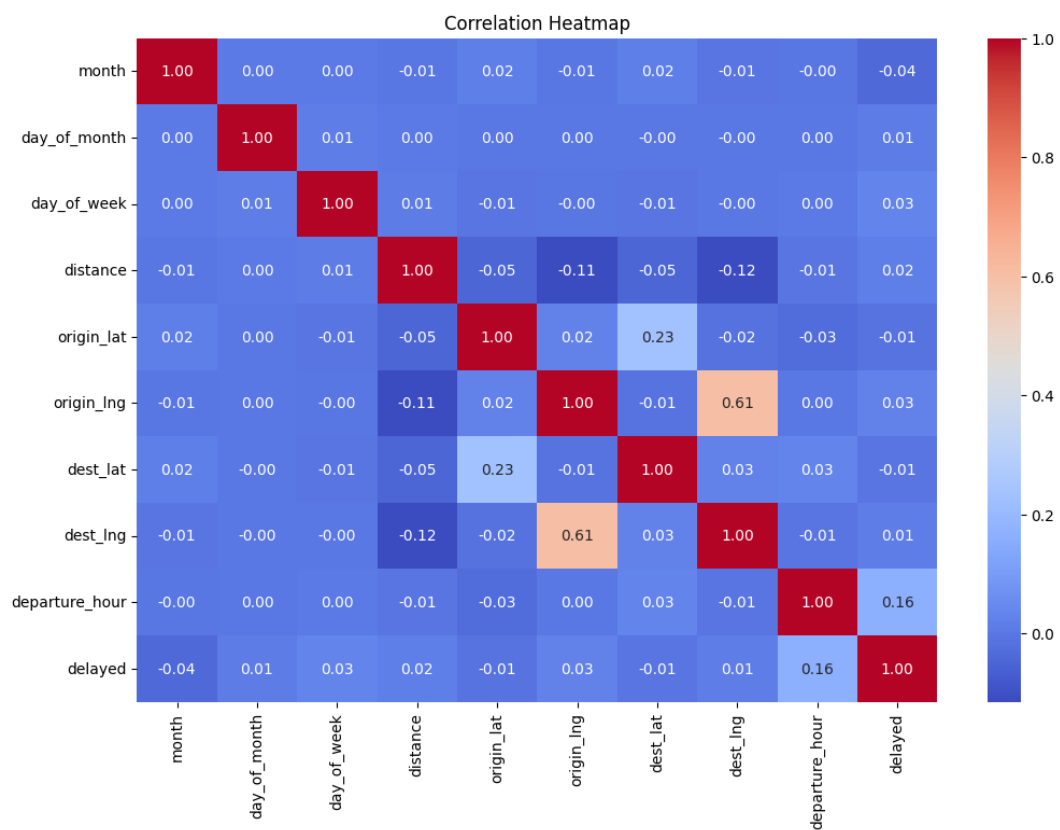
plt.figure(figsize=(12,8))

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Heatmap')

plt.show()

```



Depends on the heat map most features have a weak linear correlation with delay field, that means the flight delay are influenced by many factors (features) not only one factor.

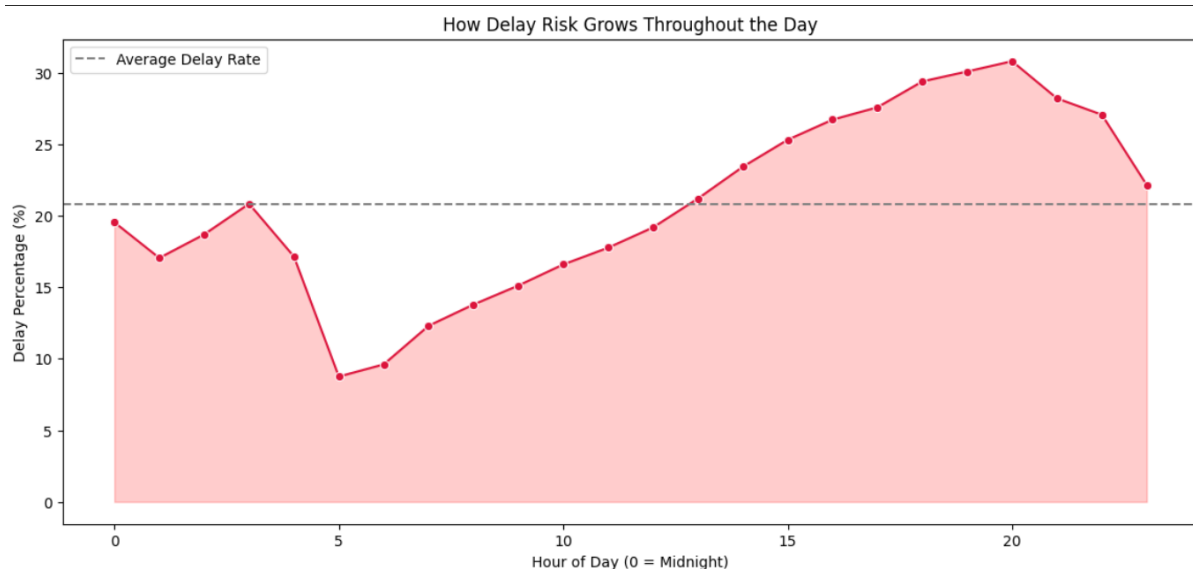
The strongest relation is the one between departure hour with delay, but is also weak one because the value is 0.16.

But as what i say before that the time of flight delay in day affect the flights after it so thats whay it is stronger a bit than the others.

**To prove what I said we will check the delay perwentage throw the day**

```
hour_delay = df.groupby('departure_hour')['delayed'].mean() * 100

plt.figure(figsize=(14,6))
sns.lineplot(x=hour_delay.index, y=hour_delay.values, marker='o', color='crimson')
plt.axhline(y=df['delayed'].mean()*100, color='gray', linestyle='--', label='Average Delay Rate')
plt.fill_between(hour_delay.index, hour_delay.values, alpha=0.2, color='red')
plt.title('How Delay Risk Grows Throughout the Day')
plt.xlabel('Hour of Day (0 = Midnight)')
plt.ylabel('Delay Percentage (%)')
plt.legend()
plt.show()
```



The graph shows clearly the increase or growth in delays start from 6 - 7 Am.

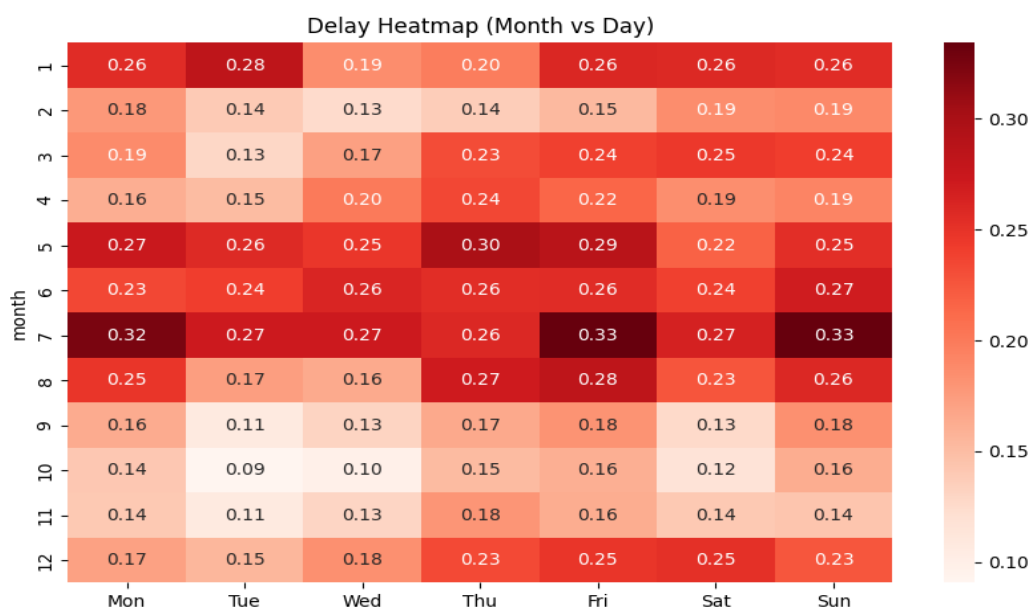
This pattern support our concept of delay in a period of day so when the delays start in the mornings it make more delays in the afternoon, evening and night and increase the percentage of delay in them.

## 10.9 Delay Heatmap (Month vs Day)

```
pivot = pd.pivot_table(
    df,
    values='delayed',
    index='month',
    columns='day_of_week',
    aggfunc='mean'
)

# Rename columns from numbers to day names
pivot.columns = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

plt.figure(figsize=(10,6))
sns.heatmap(
    pivot,
    annot=True,
    cmap='Reds',
    fmt='.2f'
)
plt.title('Delay Heatmap (Month vs Day)')
plt.show()
```



Depends on what we see the highest values is for in (Monday, Friday and Sunday) in July

First why the month is July because it is the peak summer travel season in US, because on school summer vacation and some holidays in US in that month like (Independence day).

**Monday** The travelers return from their weekend trips in monday and the bussiness travelers flight out in monday to start their work week also the delayed of sunday flights carry on monday

**Friday** Start of the weekend so people will travel, and the bussiness travelers fly back to home after work days

**Sunday** The end of the weekend so everyone come back home to start their weekend and most of the return flights be afternoon and evening also the delayed of saturday night carry on the sunday morning.

## 11. References

- [1] B. Zou and M. Hansen, "Flight delay impact on airfare and flight frequency: A comprehensive assessment," *Transportation Research Part E: Logistics and Transportation Review*, vol. 69, pp. 54–74, 2014. doi: 10.1016/j.tre.2014.05.016. Available: [https://www.researchgate.net/publication/263472147\\_Flight\\_delay\\_impact\\_on\\_airfare\\_and\\_flight\\_frequency\\_A\\_comprehensive\\_assessment](https://www.researchgate.net/publication/263472147_Flight_delay_impact_on_airfare_and_flight_frequency_A_comprehensive_assessment)
- [2] Y. Tang, "Airline Flight Delay Prediction Using Machine Learning Models," *2021 5th International Conference on E-Business and Internet (ICEBI)*, Singapore, 2021. doi: 10.1145/3497701.3497725. Available: <https://dl.acm.org/doi/10.1145/3497701.3497725>
- [3] I. Hatipoglu and O. Tosun, "Predictive Modeling of Flight Delays at an Airport Using Machine Learning Methods," *Applied Sciences*, vol. 14, no. 13, 2024, Art. no. 5472. doi: 10.3390/app14135472. Available: <https://www.mdpi.com/2076-3417/14/13/5472>
- [4] S. A. A. AlBassam and D. N. AlShahrani, "Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy," *PLOS One*, vol. 20, no. 1, 2025, Art. no. e0335141. doi: 10.1371/journal.pone.0335141. Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0335141>
- [5] J. Pineda-Jaramillo, C. Muñoz, R. Mesa-Arango, C. González-Calderón, and A. Lange, "Integrating multiple data sources for improved flight delay prediction using explainable machine learning," *Results in Tourism, Business and Management*, vol. 15, 2024, Art. no. 101161. doi: 10.1016/j.rtbm.2024.101161. Available: <https://www.sciencedirect.com/science/article/pii/S2210539524000634>
- [6] M. Dai, "A hybrid machine learning-based model for predicting flight delay through aviation big data," *Scientific Reports*, vol. 14, Art. no. 4603, Feb. 2024, doi: 10.1038/s41598-024-55217-z. Available: <https://www.nature.com/articles/s41598-024-55217-z>
- [7] *ResearchMate*, "Logistic Regression," [Online]. Available: <https://researchmate.net/logistic-regression/>. Accessed: May 1, 2026.

[8] 365 Data Science, “Decision Trees in Machine Learning,” 365 Data Science Tutorials. [Online]. Available: <https://365datascience.com/tutorials/machine-learning-tutorials/decision-trees/> . [Accessed: May 1, 2026].

[9] GeeksforGeeks, “Random Forest Algorithm in Machine Learning,” GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/> . [Accessed: May 1, 2026].

[10] GeeksforGeeks, “XGBoost,” *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/xgboost/> . [Accessed: May 2, 2026].

[11] M. Borsetti, “airportsdata,” GitHub repository. [Online]. Available: <https://github.com/mborsetti/airportsdata> . [Accessed: May 25, 2026].

[12] National Business Aviation Association (NBAA), “Causes of Delays,” *NBAA*, 2026. [Online]. Available: <https://nbaa.org/aircraft-operations/airspace/traffic-flow-management-tfm/concepts-of-traffic-flow-management/causes-of-delays/> . Accessed: May 25, 2026.

[13] Morgantown Municipal Airport Weather Problem, <https://wajr.com/2026/01/30/severe-weather-temporarily-suspends-mgw-flight-operations/>

[14] Morgantown Municipal Airport Weather Problem, <https://www.facebook.com/share/p/1EUMBPYtiM/>